CertyIQ

CertyIQ

Premium exam material

Get certification quickly with the CertyIQ Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates
First attempt guaranteed success.

https://www.CertyIQ.com

About CertyIQ

We here at CertyIQ eventually got enough of the industry's greedy exam paid for. Our team of IT professionals comes with years of experience in the IT industry Prior to training CertIQ we worked in test areas where we observed the horrors of the paywall exam preparation system.

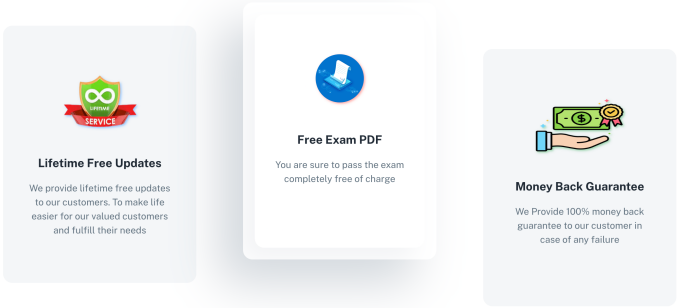
The misuse of the preparation system has left our team disillusioned. And for that reason, we decided it was time to make a difference. We had to make In this way, CertyIQ was created to provide quality materials without stealing from everyday people who are trying to make a living.

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

https://www.certyiq.com

Mail us on - certyiqofficial@gmail.com



John

October 19, 2022

★★★★★

Thanks you so much for your help. I scored 972 in my exam today. More than 90% were from your PDFs!

Dana

September 04, 2022

★★★★★

Thanks a lot for this updated AZ-900 Q&A. I just passed my exam and got 974, I followed both of your Az-900 videos and the 6 PDF, the PDFs are very much valid, all answers are correct. Could you please create a similar video/PDF for DP900, your content/PDF's is really awesome. The team did a really good job. Thank You 😊.

Ahamed Shibly

2 months ago

★★★★★

Customer support is realy fast and helpful, I just finished my exam and this video along with the 6 PDF helped me pass! Definitely recommend getting the PDFs. Thank you!

October 22, 2022

★★★★★

Passed my exam today with 891 marks. Out of 52 questions, 51 were from certyiq PDFs including Contoso case study.
Thank You certyiq team!

Henry Rome

2 months ago

★★★★★

These questions are real and 100 % valid. Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria

2 months ago

★★★★★

Simple easy to understand explanations. To anyone out there wanting to write AZ900, I highly recommend 6 PDF's.Thank you so much, appreciate all your hard work in having such great content. Passed my exam Today - 3 September with 942 score.

Anthropic

(Claude Certified Architect - Foundations)

Claude Certified Architect – Foundations (CCA-F) Certification

Total: **175 Questions**

Link: https://certyiq.com/papers/anthropic/claude-certified-architect

Question: 1 CertyIQ

After the web search agent and document analysis agent complete their tasks, the coordinator invokes the synthesis agent. However, the synthesis agent responds that it cannot complete the task because no research findings were provided. What is the most likely cause of this issue?

A. The synthesis agent needs tools that can fetch results directly from the other agents' conversation histories.

B. The synthesis agent's context window is not large enough to hold the combined outputs from both previous agents.

C. The subagents need to share a single API connection to enable automatic context sharing between invocations.

D. The coordinator did not include the outputs from the previous agents in the synthesis agent's prompt.

Question: 2 CertyIQ

When researching "renewable energy adoption," the web search agent returns recent statistics (2024: 35% adoption) while the document analysis agent extracts data from internal reports (2022: 18% adoption). The synthesis agent incorrectly flags these as contradictory sources rather than recognizing the data shows growth over time. What change would best enable the synthesis agent to correctly interpret such temporal differences?

A. Require subagents to include publication or data collection dates in their structured outputs.

B. Instruct the synthesis agent to always treat the most recent data as authoritative and place older findings in a separate historical appendix.

C. Add a conflict resolution agent that automatically discards older data when newer data exists for the same metric.

D. Configure the web search agent to only return results from the past 6 months

Question: 3 CertyIQ

Users report that final reports sometimes lack depth on specific subtopics. Investigation shows that the document analysis agent frequently identifies gaps — for instance, noting "the retrieved sources discuss API authentication but lack details on token refresh patterns" — but under the current strict pipeline, this insight isn't actionable since search has already completed. What is the most effective architectural change?

A. Have the analysis agent report specific gaps to the coordinator, which triggers targeted searches and re-invokes analysis until sufficient.

B. Add a research planning agent before the search phase that decomposes topics into specific sub-questions.

C. Have the synthesis agent attach confidence scores to each section and flag areas with insufficient coverage for manual review.

D. Have the coordinator review analysis output for gap indicators and re-invoke search with gap-informed queries when gaps are detected.

Question: 4 CertyIQ

Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analyzer had partially complete and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings. What state management approach be Information fidelity with context efficiency when restoring agent state?

A. Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts.

B. Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming.

C. Have each agent maintain its own persistent state file and reload it independently at the start of each session.

D. Index all agent outputs in a shared vector store. When resuming each agent queries the store using semantic search to retrieve relevant prior findings.

Question: 5 CertyIQ

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web search and document analysis agents didn't find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

A. Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before Invoking synthesis again.

B. Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information.

C. Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output.

D. Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.

Question: 6 CertyIQ

When analyzing complex legal cases that cite multiple precedents, the document analysis subagent processes each sequentially. A landmark case citing 12 precedents takes over 3 minutes to analyze completely. What's the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

A. Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations

B. Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents

C. Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reading single-precedent granularity

D. Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis

Question: 7 CertyIQ

Introduction monitoring shows the research phase takes longer than expected. Analysis reveals the coordinator invokes the web search subagent, waits for its response, then invokes the document analysis subagent and waits again. These tasks are independent - neither requires the other's output. How should you modify the system to run these subagents concurrently?

A. Switch both subagents to use a Haiku tier model instead of to reduce their individual execution time.

B. Create an async orchestration layer outside the agent that spawns parallel threads, each running a separate coordinator subagent pair, then aggregates results.

C. Add detailed instructions to the coordinator's system prompt explaining the performance benefits of parallel execution and requesting it invoke both subagents at the same

D. Structure the coordinator to emit both Task tool calls (for web search and document analysis) in a single response message rather than across separate conversation turns.

Question: 8 CertyIQ

Production reviews reveal inconsistent handling of uncertainty in final reports. Sometimes conflicting subagent findings are synthesized into a single confident statement (losing other times reports over hedge with excessive qualifications (becoming unhelpful). When the web search agent returns "Industry analysts estimate $50B market size (methodolo the document analysis agent returns "peer-reviewed study estimates $358 (1578, 95% CI)," the coordinator either picks one arbitrarily or produces vague statements like "the ma 6358-6508 depending on factors." What systematic approach best addresses this?

A. Configure subagents to only report findings meeting a high confidence threshold, filtering uncertain information before it reaches the coordinator.

B. Add a verification subagent that cross-references findings across sources, only passing claims to synthesis that are corroborated by at least two independent sources.

C. Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterization and methodological context.

D. Implement a confidence calibration layer that normalizes subagent uncertainty expressions to standardized probability scores (0.0-1.0), then weight-average findings by their calculated reliability scores to produce a statistically grounded synthesis.

Question: 9 CertyIQ

A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technology synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summari narrative flow. What change would most improve briefing quality?

A. Update the synthesis agent to render each content type appropriately — financial data as tables, news as prose, and technical lists as structured points.

B. Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation (such as Markdown) to facilitate more flexible rendering.

C. Standardize all subagent outputs to JSON with fields for every data type to ensure programmatic consistency across the pipeline.

D. Standardize all subagent outputs to prose summaries with a uniform character to maintain a consistent executive voice regardless of the source material.

Question: 10 CertyIQ

The coordinator agent has AgentDefinitions configured for all four specialized subagents, each with appropriate descriptions, prompts, and tool restrictions. During testing, you notice the coordinator correctly reasons about when to delegate — it generates messages like “I’ll ask the web search agent to find sources on this topic” — but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with incomplete information. Logs show no errors. What is the most likely cause?

A. Subagent context isolation means task descriptions from the coordinator don’t automatically reach subagents; you need to configure explicit context forwarding in Claude AgentOptions.

B. The coordinator’s max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified.

C. The coordinator’s allowed Tools configuration doesn’t include “Task”, so while it can reason about delegation, cannot invoke the tool required to spawn subagents.

D. The AgentDefinitions are configured correctly, but the coordinator’s system prompt doesn’t explicitly list the available subagent types, preventing the model from knowing they can be invoked.

Question: 11 CertyIQ

In production, final reports frequently contain claims without proper source attribution. Investigation shows that while the web search and document analysis agents correctly attach citations to their outputs, the synthesis agent loses track of which sources support which conclusions when combining findings. What's the most effective architectural change?

A. Add a verification step where the report generator uses semantic similarity matching against original sources to reconstruct which claims came from which documents.

B. Have the coordinator inject source identifier prefixes into text before each handoff, then parse these prefixes at report generation to reconstruct citations.

C. Maintain complete transcripts of all subagent interactions and add a citation-resolution agent to analyze logs and determine attributions before report generation.

D. Require all subagents to output structured claim-source mappings that the synthesis agent must preserve and merge when combining findings from multiple sources.

Question: 12 CertyIQ

After the web search and document analysis subagents complete their tasks, the coordinator needs to spawn the synthesis subagent to synthesize the findings. What is the correct approach for providing the synthesis subagent with the information it needs?

A. Provide the subagent with tool definitions that allow it to request outputs from other subagents via callbacks

B. Include the complete findings from both subagents directly in the synthesis subagent's prompt

C. Pass reference Identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results

D. Spawn the subagent with only a brief task description, relying on automatic context inheritance from the coordinator

Question: 13 CertyIQ

The web search agent has gathered several relevant sources for a research topic. The document analysis agent now needs to examine these sources. How does information flow between these two specialized subagents?

A. "The coordinator agent receives the web search agent's output and includes relevant findings in the prompt when invoking the document analysis agent.

B. The agents communicate through an event-driven message queue, with the document analysis agent subscribing to web search completion events.

C. The web search agent directly invokes the document analysis agent, using the discovered sources as parameters.

D. Both agents access a shared memory store where the web search agent writes findings and the document analysis agent reads them.

Question: 14 CertyIQ

After the web search agent finds 25 sources (120K tokens of raw content), the document analysis agent extracts key insights (15K tokens), and the synthesis agent produces a coherent narrative draft (3K tokens), the coordinator must pass context to the report generation agent for the final output with proper source citations. What context-passing strategy provides the best balance of completeness and efficiency?

A. Pass only the synthesis draft and have a separate post-processing pipeline match claims to sources and insert citations after the report is generated.

B. Pass the full accumulated context from all prior agents.

C. Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and ant Irant excerpts.

D. Pass a condensed summary of all prior stages that preserves the main findings and attributes them to sources by name only.

Question: 15 CertyIQ

Your search products tool queries an external catalog API that returns paginated results (50 items per request). Production logs show queries frequently match 200+ products, and the design that auto-fetches all pages causes 15-20 second delays. How should you redesign the pagination handling?

A. Create separate search products and fetch more results tools for pagination.

B. Implement server-side relevance ranking and return only the top 50 most relevant items.

C. Add a max pages parameter (default: 2) that controls how many pages are fetched internally.

D. Return the first page with total match count and cursor for additional pages.

Question: 16 CertyIQ

Your search Flights tool calls an external airline API that occasionally returns a 503 Service Unavailable error. What is the most effective way to handle this error in your tool implementation?

A. Return an empty flight list as if the search succeeded but found no matching flights.

B. Log the error internally and return an empty response, letting the model continue without the flight data.

C. Return an error message in the tool result explaining the service is temporarily unavailable.

D. Automatically retry the request up to five times with exponential backoff before returning results to the agent.

Question: 17 CertyIQ

Your MCP server implements a check_availability tool that queries an external calendar API. During testing, you encounter three error conditions:
(1) the tool is called with a malformed request, missing the required user_email parameter
(2) the calendar API returns a 404 because the specified user doesn't exist in the calendar system
(3) the calendar API returns a 503 because the service is temporarily unavailable.
How should each error be reported according to MCP's error handling design?

A. Report all three as tool results with isError: true

B. Report errors 1 and 2 as JSON-RPC protocol errors, report error 3 as a tool result with isError: true

C. Report error 1 as a JSON-RPC protocol error, report errors 2 and 3 as tool results with isError: true

D. Report all three as JSON-RPC protocol errors.

Question: 18 CertyIQ

Your documents (query) tool returns results as "Found 3 documents: Q2 Budget Proposal, Q2 Budget Forecast, Annual Review". You want the agent to document (4, multi) and doc (24, multi). What return format would best enable these multi-step workflows?

A. URLs that users can click to open the document in their browser.

B. Structured data containing document IDs and metadata for each result.

C. A JSON array of document titles extracted from the search results.

D. More detailed human-readable descriptions including the size and authors.

Question: 19 CertyIQ

Your agent has access to 50+ specialized API connectors for different external services. As the connector library grew, tool selection accuracy dropped to 58%. You design a search_connectors(description) tool that finds matching connectors, but in testing agents frequently skip searching and call connectors directly (often incorrectly), or search select wrong connectors from the filtered results.
How should you design the tool composition pattern to address both issues?

A. Design connectors with built-in compatibility validation that return descriptive errors for mismatched requests.

B. Design a find_and_execute(description, params) composite tool that searches and immediately executes the best matching connector.

C. Enhance all connector descriptions with detailed usage samples, edge cases, and input requirements. Add few-shot examples showing the correct search-then-use workflow.

D. Design search_connectors to dynamically add matched connectors to the agent's available tools. Connectors start unavailable and persist once discovered.

Question: 20 CertyIQ

Your publish article tool calls an external CMS API that occasionally returns transient errors (network timeouts, 503s) and non-transient errors (403 permission denied, 422 validation failure).
Currently, every error is returned directly to the agent, which leads to the agent retrying non-transient errors and wasting turns on failures that will never succeed. How should you partition error-handling responsibility between the tool implementation and the agent?

A. Handle all errors inside the tool: Implement retries with exponential backoff for every error type, and only surface a failure to the agent after a fixed number of retry attempts have been exhausted.

B. Handle transient errors (timeouts, 503s) with automatic retries inside the tool implementation, and surface non-transient errors (permission denied, validation fallures) to the agent with descriptive messages so it can take corrective action.

C. Surface all errors to the agent immediately with detailed context, and let the agent decide which errors to retry and how many times-keeping the tool implementation stateless and simple.

D. Implement a universal error handler that catches all exceptions and returns a generic "tool unavailable-try again later" message, shielding the agent from error complexity.

Question: 21 CertyIQ

Your remove_team_member tool uses a dry_run: boolean parameter for previewing impacts before execution. Production monitoring shows the agent bypasses the preview step in 15% of calls by calling with dry_run=false directly. You need to ensure every removal is preceded by a preview that the user explicitly confirms. What is the most reliable approach?

A. Add server-side validation that permits dry_run=false only when a dry_run=true call with identical parameters occurred within the past 60 seconds.

B. Replace with two tools: preview_remove_member returns impact details and a single-use confirmation token; execute_remove_member requires that token, binding execution to the specific previewed action.

C. Annotate the tool as requiring confirmation and configure the orchestration layer to prompt the user for approval before forwarding any calls to annotated tools.

D. Add detailed instructions and few-shot examples to the tool description requiring the agent to always call with dry_run=true first and wait for user confirmation before calling with dry_run=false.

Question: 22 CertyIQ

Your expense reimbursement agent processes employee requests using a process reimbursement tool. Company policy requires that reimbursements above $500 must be approved before funds are disbursed. The agent handles hundreds of requests daily, and you need the threshold enforcement to be tamper-proof regardless of how the agent is prompted ensures the $500 approval threshold cannot be bypassed?

A. The process reimbursement tool accepts an approved by manager parameter. The system prompt instructs the agent to only set this to true after confirming that a manager approved the request. A nightly audit script reviews all reimbursements where approved by manager was set to true.

B. Provide two tools: auto reimburse (hard-coded limit of $500) and manager approval. Include detailed system prompt instructions telling the agent to check the amount and use the appropriate tool. Add a Post ToolUse hook that logs which tool was called for auditing.

C. The process reimbursement tool accepts amount and details, and internally enforces the threshold; amounts <$500 are auto-disbursed and the tool returns a success confirmation. Amounts >$500 cause the tool to create a pending approval request and return a status indicating manager review is pending.

D. Implement the threshold check in a PreToolUse hook that inspects the amount parameter before process reimbursement executes. If the amount exceeds $500, the hook modifies the context to add a requires approval: true flag, which the tool checks before disbursing.

Question: 23 CertyIQ

Your order management system requires tools for three distinct operations: issuing refunds (requires amount and reason), canceling orders (requires reason), and res (requires shipping address). Each operation shares an order id parameter but has different additional requirements. You notice during testing that with your current frequently omits required parameters or includes irrelevant ones. What design change will most effectively improve parameter accuracy?

A. Split into three separate tools (each defining only the parameters required for that specific operation.

B. Keep one unified tool with all parameters marked optional, but add few-shot examples in the system prompt showing correct parameter combinations for each operation.

C. Keep one unified tool but add JSON Schema if-then-else conditionals to enforce that parameters like amount are required only when the operation type is "refund".

D. Keep one unified tool with a nested operation object parameter whose internal structure varies by operation type, documented in the tool description.

Question: 24 CertyIQ

Your portfolio value tool returns the total value of a user's investment portfolio. You're deciding between returning a structured JSON object with explicit fields versus returning information as a formatted text string. What is the primary advantage of using structured output with defined fields?

A. Structured JSON consumes significantly fewer tokens than natural language, substantially reducing API costs.

B. The agent can reliably extract specific values without parsing free form text, reducing errors in subsequent operations.

C. Structured JSON is processed deterministically by the model, significantly improving accuracy when extracting values.

D. JSON schemas automatically validate that the underlying API returned correct data before the agent processes it.

Question: 25 CertyIQ

Your scheduling agent uses get_available_slots(date, provider_id) to retrieve open appointment times, then book_appointment(provider_id, slot_time, patient_id) to reserve a slot. tickets show that 15% of booking attempts

fall with "slot no longer available" because another user booked the slot between the availability check and the booking call. How should you r these tools?

A. Modify book_appointment to return detailed failure information including currently available alternative slots when the requested slot is unavailable, enabling the agent to retry with a di time.

B. Keep both tools but add retry logic to the agent's system prompt, instructing it to call get_available_slots again and select a different time if booking fails.

C. Add a hold_slot(provider_id, slot_time) tool that creates a 60 second temporary reservation, requiring the agent to call it between checking availability and booking.

D. Combine both tools into a single find_and_book_appointment that atomically checks availability and books, returning either the confirmed booking or available alternatives.

Question: 26 CertyIQ

Your agent has a log_workout tool that accepts exercise_type (string), value (number), and measurement (string). Production monitoring shows the agent frequently passes mismatched combinations-using measurement: "reps" for cardio exercises like running, or measurement: "miles" for strength exercises like bench press. Your exercises naturally divide into two categories: cardio (measured in time or distance) and strength (measured in reps and sets). 23% of tool calls have invalid combinations. What approach would most effectively reduce these errors?

A. Implement server-side validation returning descriptive errors for invalid combinations, allowing the agent to retry with corrections.

B. Add enum constraints on measurement limiting values to "minutes", "miles", "reps", or "sets" to prevent arbitrary measurement strings.

C. Add explicit examples to the tool description showing valid combinations (e.g., "For running: use minutes or miles. For push-ups: use reps") with constraints for each exercise category.

D. Split into log_cardio_workout (with duration_minutes or distance_miles parameters) and log_strength_workout (with reps and sets parameters).

Question: 27 CertyIQ

Your MCP server includes archive_file(file_id) and delete_file(file_id) tools. Production logs show the agent calls delete_file when users ask to "remove old backups," policy requires archiving backup files. Both tools currently have minimal descriptions: "Archives a file" and "Deletes a file." Which change most directly improves tool selection?

A. Add a confirmation step that requires users to type "CONFIRM DELETE" before delete_file executes.

B. Implement server-side validation that rejects delete_file calls for files tagged as backups, returning an error message suggesting archive_file.

C. Expand tool descriptions to clarify use cases, adding guidance like "Do not use for backup files" to delete_file.

D. Add few-shot examples to the system prompt demonstrating that requests involving "backup" or "old" should use archive_file.

Question: 28 CertyIQ

Your CRM agent's delete_contact tool handles requests like "delete the duplicate entry for Acme Corp." The database contains similarly named records (e.g., "Acme Corp," "Acme Corporation," "ACME Corp Inc."), and analytics show 8% of deletions are reversed within 24 hours due to misidentified records. Users have also complained that the current multi-step confirmation flow adds too much friction to routine cleanup tasks. Which approach most effectively reduces the error rate while maintaining workflow efficiency?

A. Present matched records with differentiating fields and require single-click confirmation of the intended target before executing deletion.

B. Require users to supply the exact record ID from the CRM Interface rather than using natural language references to contact names.

C. Deploy automated duplicate detection that identifies and merges probable duplicates, removing the need for manual deletion requests.

D. Implement soft-delete with a 30-day recovery window so users can undo mistakes without slowing down the deletion workflow.

Question: 29 CertyIQ

After Implementing tool use with strict schema definitions, JSON syntax errors are eliminated, but 5% of extractions still have valid JSON with empty arrays or null values for required fields like citations and methodology. Spot-checking reveals that source documents contain this information, but in varied formats — Inline citations vs. bibliographies, methodology sections vs. details embedded in Introductions. What's the most effective way to address these failures?

A. Modify your schema to make citations and methodology optional, and flag Incomplete records for manual review rather than falling validation.

B. Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model falls to extract.

C. Add few-shot examples demonstrating extractions from documents with varied structures — showing how to identify citations in different formats and locate methodology details across section types.

D. Implement retry logic that re-sends requests when validation detects empty required fields.

Question: 30 CertyIQ

The system processes product reviews using tool use with a defined schema: rating (integer 1-5), pros (string array), cons (string array), and overall_sentiment (enum: positive, ne
Testing reveals two issues with brief or ambiguous reviews (-20% of the dataset): (1) for reviews like "Great product!", Claude fabricates specific pros and cons rather than Indica
Information isn't explicitly stated, and (2) for sarcastic reviews like "Well that was.. interesting", Claude picks sentiment arbitrarily since there's no option for ambiguous cases. W
modification best addresses both issues?

A. Make pros and cons optional fields, and add "neutral" and "unclear" to the sentiment enum

B. Allow empty arrays for pros/cons as valid output, and add "unclear" ss the sentiment enum

C. Add an extraction_confidence field (0.0-1.0) for each value, and filter outputs where any confidence falls below a threshold.

D. Allow null values for pros/cons, and add "unclear" to the sentiment earum.

Question: 31 CertyIQ

Your extraction system implements automatic retries when validation fails. On each retry, the specific validation error is appended to the prompt. This retry-with-error-feedback approach resolves most failures within 2-3 attempts. For which failure pattern would additional retries be LEAST effective?

A. The model extracts "et al." for co-authors when the full list exists only in an external document not in the input

B. The model extracts citation counts as locale-formatted strings ("1234") when the schema requires integers

C. The model extracts dates as ISO 8601 datetime strings ("2003-03-15T00:00:00Z") when the schema requires only the date portion.

D. The model extracts keywords as a nested object organized by category when the schema requires a flat array of strings

Question: 32 CertyIQ

Your invoice extraction uses tool use with strict JSON schemas. JSON syntax errors never occur, but 12% of extractions fail semantic validation--for example, line Item amounts don't extracted total, or vendor IDs don't match valid formats. These failures currently route to manual review. What's the most effective approach to reduce manual review volume while m accuracy?

A. Retry the extraction up to 3 times when validation fallis, accepting the first result that passes validation.

B. Implement post-processing logic that automatically corrects common amors, such as recalculating totais from line items when sums don't match.

C. When validation falls, make a follow-up request with the document, extraction, and validation errors for model correction.

D. Add stricter schema constraints with detailed field descriptions to prevent the model from generating invalid values initially.

Question: 33 CertyIQ

Your team is extracting structured data from 50,000 legacy legal contracts under a two-week deadline. Initial testing with 500 sample documents shows 82% pass JSON schema first attempt, while the remaining 18% fall due to diverse issues — missing required fields, malformed dates, and incorrectly identified parties. Documents that fail typically need refinements targeting their specific failure modes before extraction succeeds. Which batch processing strategy is the most cost-efficient while still meeting the deadline?

A. Split documents into 10 sequential batches of 5,000 each, analysing results and refining prompts between batches to improve extraction quality progressively.

B. Submit all 50,000 documents via batch API, then submit failed extractions in successive batches — refining prompts between each batch — until all documents pass validation.

C. Use the real-time API for all 50,000 documents since the batch API's 24-hour processing window creates unacceptable deadline risk.

D. Process 2,000 sample documents via real time API to identify failure patterns and refine prompts, then batch process all 50,000 with the optimized prompts.

Question: 34 CertyIQ

Your extraction pipeline processes contracts that frequently include amendments. When a contract contains both original terms and later amendments (e.g., original clause specifies "30-day payment terms" while Amendment 1 changes this to "45 days"), the model inconsistently extracts one value or the other with no indication of which applies. What's the most effective approach to improve extraction accuracy for documents with amendments?

A. Preprocess documents with a classifier that identifies and removes superseded sections before the main extraction step.

B. Implement post-extraction validation using pattern matching to detect amendments and flag those extractions for manual review.

C. Redesign the schema so amended fields capture multiple values, each with source location and effective date.

D. Add prompt instructions to always extract the most recent amendment value and ignore superseded original terms.

Question: 35 CertyIQ

Your system must extract event details from calendar invitations and output JSON that strictly conforms to a schema with fields for title, date, time, location, and attendees. Downstream reject any malformed or non-conformant JSON. What approach provides the most reliable schema compliance?

A. Define a tool with your target schema as input parameters and have Claude call it with the extracted data.

B. Pre-fill Claude's response with an opening brace to force JSON output, then complete and parse the response.

C. Append instructions like "Output only valid JSON matching the schema exactly" and implement retry logic to re-prompt when JSON parsing fails.

D. Include detailed JSON formatting instructions and the target schema in your prompt, then parse Claude's text response as JSON.

Question: 36 CertyIQ

Your schema includes a skills: string[] field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract skills mentioned." What's the most effective improvement?

A. Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."

B. Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.

C. Enrich the schema to [scondidering] to capture extraction metadata.

D. Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.

Question: 37 CertyIQ

Your pipeline uses a tool called extract_metadata with a JSON schema for paper details. You've also defined lookup_citations and verify_doi tools for enrichment. During testing, you notice that when users include requests like "extract the metadata and tell me how cited it is," Claude sometimes calls lookup_citations first, which fails because it needs the DOI that extract_metadata would provide. What's the most effective way to ensure structured metadata extraction happens first?

A. Set tool choice to ("type": "tool", "name": "extract_metadata") and process the enrichment requests in subsequent turns after receiving the extracted metadata.

B. Set tool choice to "any" so Claude must use a tool, combined with system prompt instructions prioritizing extract_metadata.

C. Set tool choice to ("type": "tool", "name": "extract_metadata") for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur.

D. Set tool choice to "auto" and reorder the tool definitions so extract_metadata appears first in the tools array, since Claude prioritizes earlier-listed tools.

Question: 38 CertyIQ

Your system has been operating with 100% human review for 3 months. Analysis shows that extractions with model confidence >90% have 97% accuracy overall. To reduce reviewer workload, you plan to automate high-confidence extractions. Before deploying, what validation step is most critical?

A. Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data.

B. Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.

C. Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors.

D. Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports.

Question: 39 CertyIQ

Your extraction system uses tool_use with a JSON schema containing 12 fields and detailed descriptions, totaling approximately 2,500 tokens for the complete tool definition. Processing documents under 150K tokens yields 98% accuracy. For documents between 175-190K tokens, accuracy drops to 71%, with information from the final third consistently missed. The model's context window is 200K tokens. What is the most likely cause?

A. Tool definitions consume input context tokens. Combined with system prompts and document content, the total approaches the context limit, degrading end-of-document processing.

B. Very long documents exceed the model's effective attention span regardless of context limits, causing accuracy degradation for content farther from the prompt instructions.

C. The model distributes attention proportionally across input length, causing fields mentioned only once near the document's end to receive insufficient processing focus.

D. Schemas exceeding 8-10 fields increase decision complexity during parameter generation, reducing extraction accuracy independent of document length.

Question: 40 CertyIQ

Your extraction pipeline processes invoices and extracts line items, subtotals, tax amounts, and grand totals. During evaluation, you discover that in 18% of extractions, the sum of extracted line item amounts doesn't match the extracted grand total — sometimes due to OCR errors in the source document, sometimes due to extraction mistakes by the model. Downstream accounting systems reject records with mismatched totals. What's the most effective approach to improve extraction reliability?

A. Add a "calculated total" field where the model sums extracted line items alongside a "stated_total" field. Flag records for human review when values differ.

B. Extract line items and totals independently, then use a separate validation model to reconcile discrepancies by determining which extracted values are most likely correct.

C. Add few-shot examples demonstrating invoices where extracted line items sum correctly to the stated total, encouraging the model to produce mathematically consistent extractions.

D. Implement post-processing that automatically adjusts line item amounts proportionally when their sum doesn't match the stated total.

Question: 41 CertyIQ

Your extraction system processes two document types: standard monthly reports (archived after processing) and urgent exception reports (must trigger business alerts within 30 minutes of receipt). Both use the same JSON schema. You want to minimize API costs while meeting latency requirements. How should you architect the processing pipeline?

A. Submit all documents to the Batch API with custom ids for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.

B. Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.

C. Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.

D. Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.

Question: 42 CertyIQ

The extraction pipeline receives documents of varying types — some are invoices, others are contracts, and some are receipts. You've defined separate extraction tools, each with its own schema tailored to the document type. During testing, you observe that with tool_choice: "auto", Claude sometimes returns conversational text instead of

calling an extraction tool, causing downstream parsing failures. You need guaranteed structured output without knowing the document type in advance. What's the most effective approach?

A. Consolidate all document types into a single unified-schema extraction tool and force that tool.

B. Keep tool_choice: "auto" with system prompt instructions requiring tool use.

C. Set tool_choice: "any" with all extraction tools defined.

D. Add a preliminary classification call, then make a second call with tool_choice forced to the identified extraction tool.

Question: 43 CertyIQ

Monitoring shows 12% of extractions fall Pydantic validation with specific errors like "expected float for quantity, got '2 to 3". Retrying these requests without modification produces failures. What's the most effective approach to recover from these validation failures?

A. Set temperature to 0 to eliminate output variability and ensure consistent formatting

B. Send a follow-up request including the validation error, asking the model to correct its output

C. Pre-process source documents to standardize problematic formats before sending them for extraction

D. Implement a secondary pipeline using a larger model tier to reprocess documents that fail validation

Question: 44 CertyIQ

After three months of weekly sessions, your conversation history has grown to 85,000 tokens. When users ask "What did we conclude about the theme of isolation?", the assistant provides generic literary analysis rather than referencing the group's specific insights from earlier sessions. Discussions often build on previous meetings' conclusions, so maintaining narrative context is important. What's the most effective approach?

A. Add structured XML tags to mark significant discussion conclusions throughout the conversation history.

B. Use semantic embedding to index the full conversation history and retrieve only relevant past exchanges for each user query, replacing the linear conversation format with retrieved segments.

C. Implement rolling window truncation to keep only the most recent 25,000 tokens.

D. Implement progressive summarization where older conversation blocks are replaced with concise summaries that explicitly extract key conclusions, decisions, and recurring themes, keeping recent exchanges verbatim.

Question: 45 CertyIQ

After a 40-minute session helping plan a dinner party, the conversation has grown to 78,000 tokens. The history includes: (1) the user mentioning a guest has a severe shellfish allergy, (2) measurements for scaling recipes to 8 servings, (3) the user's clarification that "room temperature butter" means 68°F in their kitchen, and (4) general back-and-forth about meal timing and presentation. You need to implement context management before the window limit is reached. What approach best balances information preservation with token reduction?

A. Summarize the entire conversation history into a concise summary capturing main topics discussed, then append new messages going forward.

B. Implement a sliding window retaining only the most recent 20,000 tokens relying on users to re-state important information when relevant.

C. Store the full conversation externally and use semantic search to retrieve relevant portions for each turn, loading only matching segments into context.

D. Extract critical structured data (allergies, serving counts, user-defined terms) into a compact reference section, summarize general discussion, and retain recent exchanges verbatim.

Question: 46 CertyIQ

You're Implementing a feature where users refine their playlist preferences through multiple conversation turns. After deploying, you notice Claude's responses don't reflect what us earlier in the same conversation — for example, a user says they love jazz, but two messages later Claude asks what genres they enjoy. What is the most likely cause?

A. The model's context window has been exceeded by the conversation length

B. The Claude API requires a session_id parameter that you haven't configured

C. Claude requires a vector database connection to maintain conversation memory

D. Your application isn't including prior messages in the messages array

Question: 47 CertyIQ

Your fitness coaching assistant uses a system prompt with detailed conditional logic: "If the user mentions being a beginner, provide step-by-step form instructions. If they use term 'progressive overload' or 'superset', respond concisely. If they ask about injury history, always recommend consulting a physician." During evaluation, you find the assistant correct explicit expertise declarations but struggles when users don't clearly state their level-often defaulting to overly detailed responses regardless of contextual cues like technical te
Which change to the system prompt would most directly address this failure to pick up on implicit expertise signals?

A. Replace most conditionals with a general principle: "Adapt explanation depth to match user expertise, mirroring their terminology." Keep only the safety-critical conditional abou consultations.

B. Add more conditional branches to cover additional expertise signals, such as "If user mentions specific rep ranges or asks about periodization, treat as advanced."

C. Implement a pre-conversation intake that asks users to rate their experience level, then inject that rating into the system prompt as context for all subsequent responses.

D. Add an explicit instruction for the model to ask a clarifying question about experience level whenever the user's expertise isn't immediately clear from their first message.

Question: 48 CertyIQ

During initial testing, you notice that Claude doesn't seem to remember vocabulary words from earlier in the conversation. When a student asks "Can you quiz me on those words?", responds as if no words have been discussed. What is the most likely explanation?

A. Your system prompt needs explicit instructions telling Claude to remember information from earlier turns.

B. You're not including prior messages in each API request — the stateless API doesn't retain conversation history.

C. You need to enable conversation persistence by passing a session ID parameter with each API call.

D. The model's context window has filled up, causing earlier conversation content to be dropped.

Question: 49 CertyIQ

Your home renovation planning assistant uses a system prompt defining an expert contractor persona with specific guidelines: always ask about budget, suggest alternatives at multiple price points, and confirm timeline requirements. During testing, responses follow these guidelines for turns 1-4, but by turn 7, the assistant gives generic advice without asking about budget or timeline. The conversation totals only 2,500 tokens. What is the most likely cause?

A. System prompts only establish initial behavior and don't persist across all turns.

B. The system prompt is only sent with the first API request.

C. The assistant's accumulated responses are diluting the system prompt's influence.

D. The model's attention on system prompt instructions naturally weakens as turns accumulate.

Question: 50 CertyIQ

Users report that during extended conversations, the AI loses track of specific topics, examples, and preferences they mentioned earlier in the session. Your current implementation uses a sliding window that keeps only the most recent 25 message pairs to stay within context limits. What's the most effective approach to maintain awareness of earlier conversation content while managing context size?

A. Replace the sliding window with a hybrid approach: summarize older messages while keeping recent messages verbatim.

B. Implement vector similarity search over the full conversation history, retrieving relevant past messages for each user query.

C. Increase the window size to 50 message pairs to retain more conversation history before truncation.

D. Add a separate API call each turn to summarize messages being dropped, prepending this running summary to the conversation.

Question: 51 CertyIQ

Users frequently send ambiguous requests like "book a venue for the party" without specifying date, guest count, or budget. Your evaluation shows the assistant asks an average of 4 questions before taking any action, causing 35% of users to abandon mid-conversation. However, when you reduce questions, users sometimes receive recommendations that don't preferences. What's the most effective approach to improve this trade-off?

A. Implement a structured intake form that collects all required parameters (date, guest count, budget, venue type) upfront before the assistant begins providing any recommendation

B. Configure the assistant to proceed with reasonable defaults (medium sized venue, next weekend, moderate budget) without explicitly stating these assumptions, allowing users to corrections if results don't match expectations

C. Instruct the assistant to state explicit assumptions based on conversation status proceed with recommendations while inviting corrections, and reserve clarifying questions only Irreversible actions like confirming bookings.

D. Configure the assistant to consolidate all clarifying questions into a single compound question (e.g., "What date, guest count, and budget are you considering?") to reduce the total

Question: 52 CertyIQ

During QA testing, you notice that Claude follows your system prompt guidelines consistently in the first 10-15 turns, but by turn 25-30, responses begin deviating—using informal tone when formality was specified, occasionally skipping required formatting, or providing information types the guidelines restrict. Conversation length is well within context limits (typically 30,000 tokens out of 200,000 available). What's the most effective approach to maintain consistent behavior throughout extended conversations?

A. Insert user-role messages that reinforce critical guidelines at natural conversation breakpoints, especially before complex requests.

B. Implement post-response validation that regenerates each response until it conforms to the specified guidelines.

C. Automatically start a new conversation after 20 turns, passing a summary of the prior context to maintain continuity.

D. Move behavioral guidelines from the system prompt into the first user message.

Question: 53 CertyIQ

Performance analysis reveals your context is composed of accumulated RAG results from all previous queries, which is crowding out conversation history and causing coherence degradation after 15+ turns. Which approach best addresses this issue?

A. Implement semantic deduplication to identify and remove redundant information across the accumulated RAG results and conversation turns

B. Implement a sliding window for RAG results from the last 2-3 queries while preserving conversation history

C. Shift context budget to favor RAG results while reducing conversation history allocation

D. Compress all RAG results into a consolidated summary document that updates incrementally after each retrieval

Question: 54 CertyIQ

Your music discovery assistant should consistently maintain an enthusiastic tone, explain its reasoning for each recommendation, and ask clarifying questions to better understand user preferences. You want this behavior to persist reliably across all user interactions. Where should you define these behavioral guidelines?

A. In the first assistant message, instructing Claude to follow these guidelines going forward

B. Prepended to each user message before sending to the API

C. In the system prompt

D. In environmental variables that your application passes to the API client

Question: 55 CertyIQ

During a conversation about order tracking, your external system receives a webhook indicating the user's package has shipped. The user is actively chatting and will likely send a follow-up message soon. You want the assistant to naturally incorporate this status change in its next response. What's the most effective approach?

A. Immediately send an API request with the update as a synthetic user message, generating an unsolicited assistant response.

B. Append the status update as a prefix to the next user message before calling the API.

C. Configure the assistant to call a get_order_status tool at the start of every response.

D. Add the current shipping status to the system prompt before the next API call.

Question: 56 CertyIQ

A new user's first message is "Set up my focus music," This could mean configure preferences, create a playlist, or play music immediately. Your system supports all three actions. What's the effective approach?

A. Create a new "Focus" playlist with curated tracks and notify the user it's ready.

B. Ask one clarifying question about action type: play now or configure for later

C. Play popular focus tracks Immediately and let the user redirect if needed

D. Start preference configuration by asking about genres, temps, and artists they prefer for focus.

Question: 57 CertyIQ

Users report that responses feel repetitive across turns — each message begins with phrases like "Certainly!" or "I'd be happy to help!" even deep into conversations. You want responses to feel more natural, without these repetitive openers. What's the most effective approach?

A. Implement post-processing to detect and strip common greeting phrases from response beginnings

B. Add system prompt instructions specifying phrases to avoid, such as "Never begin responses with 'Certainly' or similar affirmations"

C. Lower the temperature parameter to make response openings more deterministic and less variable

D. Append a partial assistant message with a direct response opening that the model will continue from

Question: 58 CertyIQ

Users frequently send ambiguous requests like "book a venue for the party" without specifying date, guest count, or budget. Your evaluation shows the assistant asks an average of 4 questions before taking any action, causing 35% of users to abandon mid-conversation. However, when you reduce questions, users sometimes receive recommendations that don't preferences. What's the most effective approach to improve this trade-off?

A. Implement a structured intake form that collects all required parameters (date, guest count, budget, venue type) upfront before the assistant begins providing any recommendation

B. Configure the assistant to proceed with reasonable defaults (medium sized venue, next weekend, moderate budget) without explicitly stating these assumptions, allowing users to corrections if results don't match expectations

C. Instruct the assistant to state explicit assumptions based on conversation status proceed with recommendations while inviting corrections, and reserve clarifying questions only Irreversible actions like confirming bookings.

D. Configure the assistant to consolidate all clarifying questions into a single compound question (e.g., "What date, guest count, and budget are you considering?") to reduce the total

Question: 59 CertyIQ

Your process refund tool returns two types of errors: technical errors ("503 Service Unavailable", "Connection timeout") that are transient (5% of calls), and business errors ("Order exceeds 3 day return window", "item already refunded") that are permanent (12% of calls). Monitoring shows the agent wastes 3-4 turns retrying business errors that can never succeed. Currently, both error types return only a plain text message to Claude.
What's the most effective way to reduce wasted retries while improving customer-facing response quality?

A. Add few-shot examples showing how to distinguish retriable from non-retriable errors by parsing error message text.

B. Add a check refund eligibility tool that must be called before process refund to prevent business rule violations.

C. Implement automatic retry logic at the tool level for technical errors only, passing business errors to Claude without retries.

D. Return structured error responses with retriable false for business errors and a customer-friendly explanation for Claude to use.

Question: 60 CertyIQ

Your agent has called lookup_order multiple times while investigating a customer's return requests. Each response includes 40+ fields (items, shipping details, payment inf outputs now represent the majority of the conversation's context. The customer mentions two more orders they want to discuss. What's the most effective approach before lookups?

A. Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues

B. Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details

C. Have the model generate a natural language summary of each order's key details, replacing structured

responses with prose descriptions

D. Proceed with additional lookups without modifying the existing tool output context

Question: 61 CertyIQ

During a billing dispute resolution, your agent successfully retrieves customer info via get_customer and order details via lookup_order, but when attempting to call process_refund, the tool returns a timeout error. The agent has enough information to explain the charges and verify refund eligibility, but cannot actually process the refund due to the backend failure. What approach best balances first-contact resolution with appropriate error handling?

A. Confirm the refund will be processed and close the conversation, since the system has all necessary information to complete it automatically

B. Explain the billing, confirm refund eligibility, acknowledge the system issue preventing immediate processing, and offer escalation or retry later

C. Escalate immediately to a human agent since the refund action cannot be completed

D. Implement automatic retries with exponential backoff for process_refund, keeping the conversation open until the refund is successfully processed

Question: 62 CertyIQ

Your agent is handling a billing dispute. After calling get_customer and lookup_order, it identifies that the dispute involves a promotional pricing error requiring manager approval — beyond the agent's authorization level. How should the workflow handle this mid-process escalation?

A. Persist the complete conversation and tool response history to a database, then call escalate_to_human with a reference ID.

B. Call escalate_to_human passing only the customer's original message.

C. Attempt the refund with process_refund anyway, escalating only if the system rejects the transaction.

D. Compile a structured handoff with customer details, order info, and the identified issue before calling escalate_to_human.

Question: 63 CertyIQ

Production logs reveal inconsistent error handling: when lookup_order fails, the agent sometimes retries 5+ times (wasteful when the order ID doesn't exist), sometimes escalates immediately (premature for temporary network issues), and sometimes asks users for clarification (inappropriate when the issue is a backend permission error). Investigation shows your MCP tool returns uniform error responses: "isError": true, "content": ["type": "text", "text": "Operation failed"] . The agent cannot distinguish between error types. What's the most effective improvement?

A. Enhance error responses with structured metadata: include errorCategory (transient/validation/permission), isRetryable boolean, and a description of what caused the failure.

B. Implement retry logic with exponential backoff in your MCP server for all errors, returning to the agent only after retries are exhausted.

C. Add few-shot examples to the system prompt demonstrating how to interpret error message patterns and select appropriate responses for each.

D. Create an analyze_error MCP tool the agent calls after any failure to determine the error category and recommended action.

Question: 64 CertyIQ

A customer raises three separate issues during one session: a refund inquiry (turns 1-15), a subscription question

(turns 16-30), and a payment method update (turns 31-45). At turn 48, the customer asks "What happened with my refund?" The conversation is approaching context limits. What strategy best maintains the agent's ability to address all issues throughout the session?

A. Implement sliding window context that retains the most recent 30 turns.

B. Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer.

C. Rely on MCP tools to re-fetch relevant information on demand when the customer references earller issues.

D. Summarize earlier turns into a narrative description, preserving full message history only for the active issue.

Question: 65 CertyIQ

Your update_user_profile tool accepts a user_id (required) and an optional fields_to_update object. In testing, Claude frequently omits user_id or passes incorrectly structured data. What is most critical for helping Claude understand what parameter values to provide?

A. Clear parameter descriptions explaining expected format, such as "user_id : UUID of the user to update (required)"

B. Verbose parameter names encoding format hints, such as user_id_string_uuid_format

C. Strict JSON Schema type constraints marking user_id as required and defining fields_to_update as an object type

D. Detailed error responses explaining why invalid parameter values were rejected

Question: 66 CertyIQ

Production monitoring shows your search_catalog tool fails 12% of the time: 8% are network timeouts that succeed when immediately retried, while 4% are query syntax errors from malformed user-provided filters that never succeed regardless of retry attempts. Currently, both error types are returned to the agent identically, causing it to waste turns retrying syntax errors and telling users to "try again later" for timeouts. How should you modify the tool's error handling?

A. Apply exponential backoff retry logic to all errors uniformly, returning a generic "service temporarily unavailable" message after max retries are exhausted.

B. Return all errors with a retryable boolean flag and error type details.

C. Implement automatic retry with backoff for network timeouts inside the tool; return syntax errors immediately with parameter validation details.

D. Add few-shot examples to your system prompt demonstrating how to distinguish network errors from syntax errors and handle each case appropriately.

Question: 67 CertyIQ

Users frequently send ambiguous requests like "book a venue for the party" without specifying date, guest count, or budget. Your evaluation shows the assistant asks an average of 4.2 clarifying questions before taking any action, causing 35% of users to abandon mid-conversation. However, when you reduce questions, users sometimes receive recommendations that don't match their preferences. What's the most effective approach to improve this trade-off?

A. Instruct the assistant to state explicit assumptions based on conversation history, proceed with recommendations while inviting corrections, and reserve clarifying questions only for irreversible actions like confirming bookings.

B. Configure the assistant to proceed with reasonable defaults (medium-sized venue, next weekend, moderate budget) without explicitly stating these assumptions, allowing users to provide corrections if results don't match expectations.

C. Implement a structured intake form that collects all required parameters (date, guest count, budget, venue type) upfront before the assistant begins providing any recommendations.

D. Configure the assistant to consolidate all clarifying questions into a single compound question (e.g., "What date, guest count, and budget are you considering?") to reduce the total number of conversational turns.

Question: 68 CertyIQ

Your document extraction tool uses ML models to extract invoice fields (vendor, amount, date). The models return confidence scores (0.0-1.0) for each extracted field. In production, you observe:
(1) the agent proceeds with low-confidence extractions that are incorrect 23% of the time, and (2) the agent requests unnecessary human review for 31% of extractions that were actually correct. How should you restructure the tool's output?

A. Return fields with their raw confidence scores and add detailed few-shot examples to your system prompt demonstrating how to interpret different confidence ranges and when to request human review.

B. Compute an aggregate extraction quality score across all fields and return it alongside the extracted values. Include a text summary describing the overall extraction reliability.

C. Return fields with confidence scores, plus a requires_review boolean computed using your tested confidence thresholds, along with a review_reasons array explaining which fields triggered review.

D. Return fields organized into verified and needs_verification objects based on confidence thresholds.

Question: 69 CertyIQ

Your agent includes an update_game_score tool that accepts game_date (string), home_team (string), and away_team (string) parameters. Production logs reveal recurring issues: the agent uses team nicknames instead of official names, applies inconsistent date formats, and selects the wrong game when teams have rematches in the same season. What tool interface change would effectively prevent these errors?

A. Add a season parameter to disambiguate rematches, and add a confirm_before_update flag that returns the resolved game details for the agent to verify before the score is committed.

B. Add detailed examples to the tool description showing the required date format and complete list of official team names.

C. Add enum constraints listing valid team names for both team parameters, and add a regex pattern enforcing ISO 8601 format for the date parameter.

D. Replace the three parameters with a single game_id parameter and a separate search_games lookup tool that returns matching game IDs.

Question: 70 CertyIQ

Your resource allocation tool returns a simple acknowledgment message after provisioning is requested. Users frequently approve allocations and immediately ask "how much did that cost?" or "which project was that?" - indicating they confirmed without understanding the request. What tool design change would most effectively address this?

A. Add a user_acknowledged: boolean parameter that must be set true, with instructions for the agent to only set it after the user explicitly confirms they reviewed the details

B. Implement a 60-second hold before execution completes, allowing users time to review pending allocations and cancel if needed

C. Add a detail_level parameter with options "minimal" or "comprehensive" that controls how much context the agent presents in confirmations

D. Return structured data including cost estimate, target project, resource specifications, and impact summary in the tool response

Question: 71 CertyIQ

Your conversation history includes two types of content: persistent story elements (character backgrounds, plot structure, world rules) that must remain consistent throughout, and extensive brainstorming discussion that's mostly ephemeral. After 40+ turns, you're hitting context limits and users report the assistant "forgets" established character traits, breaking narrative consistency.
Which approach best ensures persistent story elements remain available to the model while reclaiming context space?

A. Separate persistent story elements into a retained "story bible" section at context start, applying trimming or summarization only to brainstorming discussion.

B. Store all history in a vector database and retrieve semantically similar passages for each new message, replacing conversation history with retrieved chunks.

C. Apply a sliding-window approach keeping only the most recent 25 turns, relying on the model to infer earlier context from recent discussion flow.

D. Summarize the entire conversation history into a condensed synopsis every 20 turns, replacing the full history to free up tokens.

Question: 72 CertyIQ

Your conversational assistant frequently generates multiple clarifying questions when users make ambiguous requests. When a user asks "Can you help me with the report?", the assistant responds: "I'd be happy to help! Could you tell me: 1) Which report? 2) What kind of help—drafting, reviewing, or formatting? 3) What's your deadline?" User analytics show a 40% conversation abandonment rate after these multi-question responses. What's the most effective way to reduce friction while appropriately handling ambiguity?

A. Limit the assistant to one clarifying question per turn, using conversation history to accumulate answers over multiple exchanges rather than requesting everything upfront.

B. Modify the system prompt to instruct the assistant to make reasonable assumptions from available context, state those assumptions explicitly, and offer to adjust if the interpretation is wrong.

C. Add a preprocessing step using a smaller model to classify request ambiguity on a 1-5 scale, routing high-ambiguity requests to a clarification dialog and low-ambiguity requests directly to the assistant.

D. Create a lookup table of common request patterns with predefined default interpretations, having the assistant respond with those defaults without stating the assumptions made.

Question: 73 CertyIQ

Users frequently refine their search criteria mid-conversation. You notice a pattern: when users say things like "Actually, let's raise the budget to $650K" or "I'd prefer a condo now instead of a house," the assistant sometimes continues referencing the original preferences in later responses—even though the updates are clearly present in the conversation history. Context usage is only at 35% capacity. Which solution most reliably ensures the model uses the current preferences?

A. Maintain a structured state object with current preferences, update it on changes, and include it in each request.

B. Implement conversation pruning to remove turns containing outdated preferences, ensuring only current ones remain in context.

C. Include few-shot examples showing the assistant correctly acknowledging and applying preference changes in responses.

D. Add system prompt instructions emphasizing that the model should always prioritize the most recently stated preferences over earlier ones.

Question: 74 CertyIQ

Your conversational AI tutor has a 2,800-token system prompt containing teaching methodology, persona guidelines, and detailed written instructions for adapting explanations to different proficiency levels. User testing reveals that in conversations exceeding 12 turns (approximately 4,000 tokens of conversation history), the

assistant increasingly ignores the proficiency-adaptation guidelines, defaulting to intermediate-level explanations regardless of the learner's stated level. What's the most effective approach to ensure consistent adherence to these guidelines throughout extended conversations?

A. Inject a condensed reminder of the proficiency requirements into the conversation as a system message every 4-5 turns.

B. Replace the verbose proficiency guidelines with few-shot examples demonstrating appropriate responses at each proficiency level, showing concrete differences in vocabulary, complexity, and explanation depth.

C. Restructure the system prompt to place the proficiency-adaptation rules in a clearly-marked final section immediately before the conversation history begins.

D. After each assistant response, make a separate API call to evaluate whether the difficulty level matched the learner's profile, regenerating responses that don't align.

Question: 75 CertyIQ

After deploying an updated system prompt that improves response quality, users with multi-session conversations spanning several weeks report that the assistant now contradicts its earlier statements and has a noticeably different communication style. New users don't experience these issues.
What's the best approach to resolve this?

A. Regenerate summaries of existing conversations using the new prompt and replace the stored histories to align past context with current behavior.

B. Add a transition message when sessions resume explaining that the assistant has been updated and behavior may differ.

C. Version system prompts and associate each conversation with the prompt version under which it started, applying updates only to new conversations.

D. Add instructions to the new system prompt directing the assistant to maintain consistency with any prior statements in the conversation history.

Question: 76 CertyIQ

The coordinator agent has AgentDefinitions configured for all four specialized subagents, each with appropriate descriptions, prompts, and tool restrictions. During testing, you notice the coordinator correctly reasons about when to delegate—it generates messages like "I'll ask the web search agent to find sources on this topic"—but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with incomplete information. Logs show no errors. What is the most likely cause?

A. The coordinator's max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified.

B. The coordinator's allowedTools configuration doesn't include "Task", so while it can reason about delegation, it cannot invoke the tool required to spawn subagents.

C. Subagent context isolation means task descriptions from the coordinator don't automatically reach subagents; you need to configure explicit context forwarding in ClaudeAgentOptions.

D. The AgentDefinitions are configured correctly, but the coordinator's system prompt doesn't explicitly list the available subagent types, preventing the model from knowing they can be invoked.

Question: 77 CertyIQ

Production reviews reveal inconsistent handling of uncertainty in final reports. Sometimes conflicting subagent findings are synthesized into a single confident statement (losing nuance), while other times reports over-hedge with excessive qualifications (becoming unhelpful). When the web search agent returns "industry analysts estimate $50B market size (methodology varies)" and the document analysis agent returns "peer-reviewed study estimates $35B (±$7B, 95% CI)," the coordinator either picks one arbitrarily or produces vague statements like "the market may be $35B-$50B depending on factors." What systematic approach best addresses this?

A. Implement a confidence calibration layer that normalizes subagent uncertainty expressions to standardized probability scores (0.0-1.0), then weight-average findings by their calibrated confidence.

B. Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterizations and methodological context.

C. Configure subagents to only report findings meeting a high-confidence threshold, filtering uncertain information before it reaches the coordinator.

D. Add a verification subagent that cross-references findings across sources, only passing claims to synthesis that are corroborated by at least two independent sources.

Question: 78 CertyIQ

You've configured the system so that all four subagents have access to the complete set of 18 tools. During testing, agents frequently call tools outside their specialization — the synthesis agent attempts web searches, and the report generator tries to analyze documents. What is the primary cause of this poor tool selection behavior?

A. Choosing from 18 tools instead of 4-5 relevant ones increases decision complexity beyond reliable selection thresholds.

B. The agents' role descriptions in their system prompts conflict with having access to tools outside that role.

C. The tool definitions consume too much context window space, leaving insufficient room for task content.

D. The coordinator cannot track which capabilities each subagent has, leading to misrouted tasks.

Question: 79 CertyIQ

The coordinator provides detailed step-by-step instructions to the web search subagent, specifying exact search queries, source priorities, and date filters. Production monitoring reveals three issues: (1) the subagent reports "insufficient results" rather than trying alternative approaches when pre-specified searches fail, (2) research quality drops for emerging topics that don't match expected patterns, and (3) the subagent rarely surfaces valuable tangential sources. What's the most effective way to improve subagent adaptability?

A. Specify research goals and quality criteria (coverage breadth, source diversity, recency) rather than procedural steps, letting the subagent determine its search strategy.

B. Add explicit fallback directives to the detailed instructions: "If specified searches yield fewer than N results, attempt alternative query formulations before reporting failure."

C. Remove procedural details entirely, delegating with simple goals like "research X thoroughly" and relying on the subagent's general capabilities.

D. Implement a topic classification step where the coordinator categorizes requests as "well-defined" or "exploratory" and uses different instruction styles for each category.

Question: 80 CertyIQ

The synthesis agent receives summarized findings from the web search and document analysis agents, then passes a consolidated summary to the report generator. During testing, you discover the generated reports make factual claims without proper citations — the report generator cannot attribute statements to their original sources because that metadata was lost during the summarization steps. What's the most effective approach to ensure proper source attribution in the final reports?

A. Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers).

B. Have the report generator query the web search agent to re-locate sources for claims in the final report.

C. Skip summarization and pass full raw outputs from web search and document analysis directly to the report generator.

D. Instruct the synthesis agent to embed source references inline within its summary text using a consistent citation format.

Question: 81 CertyIQ

A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with revenue, margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technology areas. The synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose narrative flow. What change would most improve briefing quality?

A. Update the synthesis agent to render each content type appropriately — financial data as tables, news as prose

B. Standardize all subagent outputs to JSON with fields for claim, evidence, source, and confidence

C. Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation

D. Standardize all subagent outputs to prose summaries with inline citations

Question: 82 CertyIQ

When the agent calls lookup order and receives order details showing the item was purchased 45 days ago, how does the agentic loop determine whether to call process refund escalate to human next?

A. The agent executes the remaining steps in a tool sequence planned at the start of the request.

B. The order details are added to the conversation and the model reasons about which action to take.

C. The agent follows a pre-configured decision tree mapping order attributes to specific tool calls.

D. The orchestration layer automatically routes to the next tool based on the order's status field.

Question: 83 CertyIQ

The agent verifies customer identity through a multi-step process before resetting passwords. During testing, you notice that after the customer answers the third verification question, the agent asks them to provide their name again, as if the earlier exchange never happened. What's the most likely cause of this behavior?

A. Claude's memory retention is limited to two conversational turns by default, requiring explicit configuration to extend it.

B. The prompt lacks instructions telling Claude to remember information across multiple exchanges.

C. The verification tool is clearing the agent's internal state after each successful validation step.

D. The conversation history isn't being passed in subsequent API requests.

Question: 84 CertyIQ

When implementing your lookup_order MCP tool, the backend sometimes returns errors (e.g., "Order not found" or temporary database failures). What is the correct pattern for communicating these errors back to the agent?

A. Log the error server-side and return an empty result to avoid confusing the model

B. Throw an exception from the tool handler so the agent framework can catch and log it

C. Return the error message in the tool result content with the isError flag set to true

D. Return a success response with a "status" field indicating the error type

Question: 85 CertyIQ

Evaluation shows 94% extraction accuracy on short meeting transcripts (<30 minutes) but only 68% on longer transcripts (>60 minutes) where discussions meander and key information is scattered throughout. Transcripts of both lengths fit within the model's context window. What pattern most effectively improves accuracy on complex, lengthy documents?

A. Add few-shot examples demonstrating correct extraction from lengthy meetings with scattered Information.

B. Split lengthy transcripts Into chunks, extract from each chunk separately, then merge and deduplicate the results.

C. Upgrade to a more capable model tier for the extraction task,

D. Add a pre-extraction step where the model summarizes key discussions and conclusions before performing structured extraction.

Question: 86 CertyIQ

Your system has been running for 3 weeks and human reviewers have corrected 847 extractions. Analysis reveals a recurring pattern: when recipes use informal measurements like "a handful" or "a splash," the model either invents specific amounts or leaves fields empty — accounting for 23% of all corrections. How should you use this feedback to improve extraction accuracy?

A. Update your JSON schema to add a "measurement_type" enum field (precise/informal).

B. Add few-shot examples to your prompt demonstrating correct handling of informal measurements — extracting them verbatim rather than converting or omitting them.

C. Implement a post-processing layer that uses pattern matching to detect informal measurement phrases in source text and automatically populate values when the extraction is empty.

D. Fine-tune the model on the 847 corrected extractions.

Question: 87 CertyIQ

Production monitoring shows that follow-up queries like "summarize what we learned about market trends" consistently take 40+ seconds. Investigation reveals the coordinator spawns the synthesis subagent for each summarization request, passing 80K+ tokens of accumulated findings. The coordinator already has these findings in its context from orchestrating the research. What's the most effective way to improve response time for these follow-up summaries?

A. Enable prompt caching on the synthesis subagent to reduce the overhead of repeatedly transferring the same research findings.

B. Have the coordinator handle straightforward summarization requests directly using its existing context, reserving subagent spawning for complex analy

C. Spawn the synthesis subagent with reduced context and have it request specific findings from the coordinator on-demand.

D. Pre-generate and cache summaries at multiple granularities whenever new findings accumulate.

Question: 88 CertyIQ

The document analysis agent has a single analyze_document tool that takes a document and a free-text instruction parameter. During evaluation, requests like "extract the key financial metrics" often return narrative summaries, while "summarize the methodology" sometimes returns raw data tables. The synthesis agent reports that 35% of analysis results require re-requests with clarified instructions. What's the most effective way to improve reliability?

A. Enhance the tool description with detailed examples showing how different instruction phrasings should map to different output formats

B. Keep the single tool but add an analysis_type enum parameter requiring explicit selection between extraction, summarization, and verification modes

C. Split the generic tool into purpose-specific tools - extract_data_points, summarize_content, verify_claim_against_source - each with defined input/output contracts

D. Have the coordinator pre-classify each analysis request before passing instructions to the document analysis agent

Question: 89 CertyIQ

The system routes documents with extraction confidence below 85% to human review. A quarterly audit reveals that 12% of high-confidence extractions (>85%) also contain errors — cases where the model finds plausible-but-incorrect values. Error sources vary: comparison tables showing competitor specs, appendices referencing different product variants, and ambiguous phrasing the model misinterprets. You need a sustainable strategy to catch these high-confidence errors and measure whether improvements reduce the error rate over time. What approach is most effective?

A. Implement heuristic rules that flag documents containing comparison tables or appendices for review regardless of confidence score.

B. Implement stratified random sampling reviewing a fixed percentage of high-confidence extractions weekly, enabling error rate measurement and novel pattern detection.

C. Add a verification pass that re-extracts from each high-confidence document, flagging cases where the two extraction attempts produce different results.

D. Lower the confidence threshold from 85% to 70%, routing a larger volume of extractions to human review.

Question: 90 CertyIQ

Your extraction uses tool use with a JSON schema where property_type is defined as an enum: ['house', 'apartment', "condo", "townhouse"]. After deployment, 8% of extractions fall schema validation. Investigation reveals listings mention many uncommon property types — "studio", "loft", "duplex", "mobile home", "tiny house", "converted warehouse" — and new types continue appearing regularly. What's the most effective long-term solution?

A. Add an "other" value to your enum with a separate property_type_detail string field for specifics when "other" is selected.

B. Change property_type from an enum to a free-form string and implement a normalization step in post-processing.

C. Add few-shot examples to your prompt demonstrating how to map unexpected property types to the closest existing enum value.

D. Continuously expand the enum to include newly observed property types and add monitoring for additional edge cases.

Question: 91 CertyIQ

Your extraction system parses e-commerce product descriptions to extract specifications like dimensions, weight, and materials into JSON. Despite having a well-defined schema, the model inconsistently extracts the "materials" field — sometimes returning "cotton blend", other times "Cotton/Polyester mix", and occasionally omitting the field when material information is clearly present in the source. What's the most effective way to improve extraction consistency?

A. Set temperature to 0 to eliminate randomness and ensure deterministic outputs

B. Switch to a more capable model tier since inconsistent extraction indicates insufficient model capability

C. Make the "materials" field required instead of optional in the schema to force the model to always extract a value

D. Add few-shot examples showing 2-3 complete Input-output pairs with standardized material description

formats

Question: 92 CertyIQ

Documents arrive continuously throughout business hours and need structured data extracted. To reduce costs, you want to use the Message Batches API (50% discount, up-to-24-hour processing window). Your SLA specifies that extraction results must be available within 30 hours of document arrival with 99.9% reliability. Which batching strategy is most appropriate?

A. Submit batches every 4 hours containing documents from that window

B. Submit batches every 6 hours containing documents from that window

C. Submit a single batch at end of day containing all documents from that day

D. Use the real-time API for all documents instead of batch processing

Question: 93 CertyIQ

Your extraction pipeline processes restaurant menus and must output structured JSON with fields for item names, descriptions, prices, and dietary tags. Some menus use inconsistent formatting — prices as "$12" vs "12.00", dietary info as icons vs text. What's the most reliable approach?

A. Extract data as-is and normalize formats in post-processing code after Claude returns.

B. Define a strict output schema and include format normalization rules in your prompt.

C. Use separate extraction calls for each field to ensure consistent handling of each type.

D. Request multiple extraction attempts per document and select the most common format.

Question: 94 CertyIQ

Your system extracts event metadata (date, location, organizer, attendee_count) from news articles using a JSON schema with all nullable fields. During evaluation, you observe the model frequently generates plausible but incorrect values for fields not mentioned in the article-for example, outputting "500" for attendee_count when the source contains no attendance information. What's the most effective way to reduce these false extractions?

A. Make all schema fleids required (non-nullable) with strict validation rules to ensure the model only outputs verifiable data.

B. Add prompt instructions to return null for any field where information is not directly stated in the source.

C. Upgrade to a more capable model tier with improved instruction-following to reduce hallucination tendencies.

D. Add a post-processing step using a second LLM call to verify each extracted value exists in the source document.

Question: 95 CertyIQ

Your research assistant helps users analyze academic papers over extended conversations. User testing reveals a recurring issue: after conversations exceed 60K tokens, users ask follow-up questions requiring precise numerical details from papers discussed earlier — sample sizes, exact p-values, specific inclusion criteria. Your current approach summarizes paper discussions after 8 turns to stay within context limits. Users report that responses to these precision-dependent questions are often hedged or inaccurate. What's the most effective architectural change?

A. Implement retrieval that re-injects relevant paper sections when the user's question suggests they need specific numerical details.

B. Maintain a structured database of key facts extracted from each paper (sample sizes, statistics, methods)

and retrieve relevant entries into context when precision-dependent questions are detected.

C. Keep source text from methodology and results sections in context permanently, while summarizing only the conversational discussion and interpretation portions.

D. Use a separate Claude call with explicit instructions to generate higher-fidelity summaries that preserve all numerical details and statistical values.

Question: 96 CertyIQ

Your fitness coaching assistant uses a system prompt with detailed conditional logic: "If the user mentions being a beginner, provide step-by-step form instructions. If they use terms like 'progressive overload' or 'superset', respond concisely. If they ask about injury history, always recommend consulting a physician." During evaluation, you find the assistant correctly adapts to explicit expertise declarations but struggles when users don't clearly state their level — often defaulting to overly detailed responses regardless of contextual cues like technical terminology. Which change to the system prompt would most directly address this failure to pick up on implicit expertise signals?

A. Add an explicit instruction for the model to ask a clarifying question about experience level whenever the user's expertise isn't immediately clear from their first message.

B. Replace most conditionals with a general principle: "Adapt explanation depth to match user expertise, mirroring their terminology." Keep only the safety-critical conditional about injury consultations.

C. Implement a pre-conversation intake that asks users to rate their experience level, then inject that rating into the system prompt as context for all subsequent responses.

D. Add more conditional branches to cover additional expertise signals, such as "If user mentions specific rep ranges or asks about periodization, treat as advanced."

Question: 97 CertyIQ

You're building a security scanning workflow. When engineers need to locate all occurrences of a dangerous function like eval() across a large codebase, which tool should your agent use for content search?

A. Use Grep to search for the pattern "eval(" across all files in the codebase.

B. Use Glob with a pattern like **/*eval to find files, then Read each matching file.

C. Use Bash to run ls -R | grep eval to recursively list files containing eval.

D. Read the project's main entry file and follow import statements to trace where eval might be used.

Question: 98 CertyIQ

agent has identified that caching logic spans 15 files including decorators, middleware, and service classes (~8,000 lines total). What's next step for building understanding while managing context constraints?

A. Use Glob to find files matching common caching patterns (cache.py, caching/), prioritize the largest files by reading them first, then check smaller files for gaps.

B. Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation.

C. Use Grep to search for "invalidate" and "expire" patterns across all files, then Read only those specific line ranges with minimal surrounding context.

D. Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations.

Question: 99 CertyIQ

Today she wants to continue that specific investigation. She's worked on three other codebases since then and

knows the session was dive". How should she resume?

A. Start fresh and re-read the same files

B. Use --resume auth-deep-dive to load that specific session by name

C. Use --continue to pick up where the most recent conversation left off

D. Use --session-id with the UUID from yesterday's session transcript file

Question: 100 CertyIQ

"You're the Lead Data Scientist/Engineer on a critical project. Something is not completely right with technical details or the data, and it negative impacts expected outputs. They need to independently develop both approaches to evaluation data... How do you manage this scenario?"

A. Start two fresh sessions, having each re-read the relevant source files before beginning.

B. Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially.

C. Resume the analysis session with fork_session enabled, creating a separate branch for each testing strategy.

D. Export the analysis session's key findings to a file, then create two new sessions that reference this file.

Question: 101 CertyIQ

A security audit requires updating your authentication library from v2 to v3. The migration guide documents breaking changes: authenticate() now returns a Promise instead of accepting a callback, the User type has restructured fields, and three deprecated methods were removed. Grep shows the library is imported in 45 files across several modules. What's the most effective approach?

A. Paste the migration guide's breaking changes into your prompt and use direct execution to update all usages across the 45 files.

B. Enter plan mode to explore library usage across modules, map affected code paths, then create a migration strategy before implementing.

C. Update the dependency version, run the test suite, and use Claude Code to fix each failure as it appears.

D. Create a custom slash command encapsulating the migration transformations, then execute it against each file without prior codebase exploration.

Question: 102 CertyIQ

During testing, you observe that in extended exploration sessions (30+ minutes), the agent starts giving inconsistent answers about context discussed earlier. Engineers report having to repeat context about modules they've already explored. What's the most effective approach to address this?

A. Create summaries of all source files before exploration begins, loading only these compressed representations into context.

B. Switch to a higher-capacity model tier to provide more context window space for accumulated exploration data.

C. Implement automatic context clearing every 15 minutes to ensure the agent starts with fresh, uncontaminated context.

D. Have the agent maintain a scratchpad file that records key findings, referencing it for subsequent questions.

Question: 103 CertyIQ

old_string parameter cannot find unique text to match — the file has repetitive docstrings, variable names, and structural patterns. What is a reliable way to complete this insertion?

A. Use Edit with an extremely long old_string capturing 30+ lines of context to guarantee uniqueness

B. Use Read to load the file, add the function at the appropriate location, then Write the updated file

C. Use Edit's replace_all parameter to target a common pattern and embed the new function in the replacement text

D. Use Bash to append the function definition to the end of the file using heredoc syntax

Question: 104 CertyIQ

You need to add a date validation check ensuring event dates are in the future. This requires adding a conditional statement to one existing function in a single file. What is the most appropriate approach?

A. Enter plan mode to analyze how the validation might impact other parts of the reservation flow

B. Start with extended thinking mode enabled to ensure thorough reasoning about the validation logic

C. Use direct execution to make the change

D. Enter plan mode first to create a detailed implementation strategy before making the change

Question: 105 CertyIQ

Your team's CLAUDE.md includes a rule: "Use 4-space indentation and always run Prettier formatting." Despite this, code reviews reveal that roughly 30% of files Claude Code generates use inconsistent formatting — sometimes 2-space indentation, sometimes missing trailing commas. Adding emphasis ("IMPORTANT: You MUST use Prettier formatting") reduces violations to about 15%, but doesn't eliminate them. What is the most effective way to ensure all generated code is consistently formatted?

A. Extract the formatting rules into a dedicated skill that Claude loads automatically when generating code, with more detailed examples of correct formatting.

B. Add a Stop hook with a prompt-based check that evaluates whether generated code follows formatting standards and prompts Claude to fix violations.

C. Configure a Post ToolUse hook with an Edit|Write matcher that automatically runs Prettier on each file Claude modifies.

D. Split the formatting rules into path-scoped .claude/rules/ files that load when Claude works on matching file types.

Question: 106 CertyIQ

You've asked Claude Code to build a PDF report generation feature. The initial implementation queries the database correctly, but the output has formatting issues: table columns are too narrow causing content truncation, dates display without proper formatting, and page break handling is incorrect. You've noticed these issues interact — changing column widths affects how dates render, and page breaks depend on content height. What's the most effective approach for iterating toward a working solution?

A. Address the column width issue first with specific measurements, verify it works, then fix date formatting within the corrected columns, then adjust page breaks— testing after each change.

B. Provide all three issues in a single detailed message with exact specifications for each, allowing Claude to address them together in one update.

C. Show Claude an example of a correctly formatted report and ask it to match that output, rather than listing the specific technical issues.

D. Start fresh with a detailed prompt specifying all formatting requirements upfront.

Question: 107 CertyIQ

You're implementing a caching layer for API responses to speed up the /products endpoint. You have a rough idea — Redis with a 5-minute TTL — but you're new to production caching and aren't sure what other considerations a robust implementation requires. What's the most effective way to start your iterative workflow?

A. Ask Claude to interview you about the caching requirements before implementing, surfacing considerations like invalidation strategies, cache layers, consistency guarantees, and failure modes.

B. Use plan mode to analyze the current/products endpoint implementation, then provide your caching requirements once Claude explains how the existing code is structured.

C. Start with a minimal request: "Add Redis caching to/products with 5-minute TTL." Add features and fix issues through follow-up prompts as problems surface during testing.

D. Write a specification with your known requirements and "TBD" markers for uncertain areas, having Claude propose solutions for each TBD as it implements.

Question: 108 CertyIQ

You're implementing a complex graph traversal algorithm with specific performance requirements and edge cases to handle (disconnected nodes, cycles, weighted edges).
You want to structure your workflow for efficient iterative refinement with Claude. What approach will most effectively enable progressive improvement across multiple iterations?

A. Write a test suite covering expected behavior, edge cases, and performance requirements before implementation. Ask Claude to write code that passes the tests, then iterate by sharing test failures with each refinement request.

B. Provide Claude with a detailed natural language specification of the algorithm, including all requirements and edge cases. Review each output manually and provide descriptive feedback on what behavior needs to change.

C. Provide Claude with a reference implementation from documentation, then ask it to rewrite the code to match your codebase style and add the required edge case handling, comparing outputs against the reference.

D. Have Claude extensively research the algorithm and create a detailed implementation plan using extended thinking, then implement the complete solution based on that plan.

Question: 109 CertyIQ

Your team is configuring MCP servers in Claude Code. You want to add a shared venue lookup server that all team members should have access to, and you personally want to add an experimental music playlist server that only you are testing. Which configuration approach correctly applies MCP server scopes?

A. Add venue server to .mcp.json and playlist server to ~/.claude.json

B. Add both servers to the project-level .mcp.json file

C. Add venue server to ~/.claude.json and playlist server to .mcp.json

D. Add both servers to your local ~/.claude.json

Question: 110 CertyIQ

Your infrastructure-as-code repository includes Terraform modules (/terraform/), Kubernetes manifests (/kubernetes/), and CI/CD pipeline scripts (/pipelines/). Each requires different conventions, but your single root CLAUDE.md has grown to 500+ lines. When developers work on Kubernetes files, Terraform-specific rules load into context unnecessarily, consuming tokens. What is the best approach to reorganize so only relevant guidance loads when editing specific file types?

A. Restructure the root CLAUDE.md into clearly labeled sections with headers (e.g., "## Terraform Conventions"), improving organization and readability.

B. Split content into subdirectory CLAUDE.md files (/terraform/CLAUDE.md, /kubernetes/CLAUDE.md), so Claude loads directory-specific guidance.

C. Keep the root CLAUDE.md and use @path/to/import syntax to modularly include tool-specific guidance files from separate documents.

D. Create files in .claude/rules/ with YAML frontmatter path-scoping (e.g., paths: ["terraform/**/*"]), loading rules only when editing matching files.

Question: 111 CertyIQ

Your team frequently migrates React components to Vue. You've written a step-by-step workflow for Claude Code to follow during each migration, and you want every developer on the team to invoke it by typing /migrate-component. The workflow should stay in sync as the team iterates on it. Where should you place the skill file?

A. In ~/.claude/skills/migrate-component/SKILL.md on each developer's machine

B. In the project's .claude/settings.json using a skillOverrides entry to register and define the workflow

C. In .claude/skills/migrate-component/SKILL.md at the project root, committed to version control

D. As a detailed instruction block in the project's root CLAUDE.md file

Question: 112 CertyIQ

A critical bug is affecting production users. Error logs show exceptions in the OrderProcessing module with a clear stack trace pointing to a specific function. You haven't worked with this module before. What's the most effective approach?

A. Start with direct execution to gather initial information, then switch to plan mode to design a comprehensive solution before implementing any changes.

B. Use plan mode to analyze the error in context of the module's design, enumerate potential root causes, and prioritize fixes systematically.

C. Enter plan mode to explore the module's architecture and dependencies before attempting any fixes.

D. Use direct execution to examine the stack trace, read the relevant code, and implement a fix once you identify the root cause.

Question: 113 CertyIQ

You're implementing a new payment processing module that must follow your project's established patterns for database transactions, error handling, and audit logging.
You've identified three existing modules that exemplify these patterns: db_utils.py, error_handlers.py, and audit_logger.py. This is a one-off integration task — these patterns are well-documented in your team wiki and don't need additional project-level documentation. What's the most effective approach?

A. Describe the patterns from the three modules in natural language in your prompt, explaining the transaction handling approach, error format, and logging conventions Claude should follow.

B. Use @references to include the three modules directly in your prompt, giving Claude concrete code examples of the patterns to follow.

C. Ask Claude to explore your codebase to find and understand the transaction, error handling, and logging patterns before generating the new module.

D. Add documentation of each pattern to your CLAUDE.md file, establishing them as project conventions that Claude will apply automatically.

Question: 114 CertyIQ

You've asked Claude to write a data migration script, but the initial output doesn't correctly handle records with null values in required fields. What's the most effective way to iterate toward a working solution?

A. Provide a test case with example input containing null values and the expected output, then ask Claude to fix it.

B. Manually edit the generated code to fix the null handling, then continue working with Claude on other parts.

C. Describe the null value problem in detail and ask Claude to regenerate the entire script with improved edge case handling.

D. Add "think harder about edge cases" to your prompt and request a complete rewrite of the migration logic.

Question: 115 CertyIQ

You've documented API error handling conventions in a CLAUDE.md file at your project root, specifying that endpoint handlers should use a custom ApiError class. After several sessions, you notice Claude Code sometimes follows these conventions and sometimes uses generic try/catch blocks with string messages. The inconsistency appears random across different coding sessions. What's the most efficient first diagnostic step?

A. Add more detailed code examples to your CLAUDE.md showing the exact ApiError usage pattern for different endpoint types.

B. Run /memory to check which memory files are loaded and verify your CLAUDE.md is included.

C. Search for conflicting instructions in ~/.claude/CLAUDE.md or ~/.claude/rules/ that might override your project conventions.

D. Create path-specific rules in claude/rules/handlers.md with YAML frontmatter scoping the error handling instructions to your API handler files.

Question: 116 CertyIQ

Your monorepo contains shared coding standards in /docs/standards/security-rules.md (for services handling user data), testing-patterns.md (for all packages), and api-conventions.md (for API-facing services). Your 15 packages are organized by feature domain (/packages/auth/, /packages/billing/, /packages/notifications/, etc.) without naming conventions indicating which handle user data or expose APIs. Package maintainers are expected to configure their own local development settings, as they understand their package's domain requirements. Currently, all package CLAUDE.md files duplicate all three standards, applying irrelevant guidance. What's the most effective approach?

A. Create claude/rules/ files for each standard with YAML frontmatter paths listing every package directory where that standard should apply.

B. Create a shared-standards.nd that uses @imports to combine all three standards, then have each package's CLAUDE.md import that combined file.

C. Put all standards in the root CLAUDE.md with override instructions like "ignore security-rules.md when working in packages that don't handle user data."

D. Use @imports in each package's CLAUDE.md to reference only the specific standard files relevant to that package, based on the maintainer's domain knowledge.

Question: 117 CertyIQ

The system processes product reviews using tool use with a defined schema: rating (integer 1-5), pros (string array), cons (string array), and overall_sentiment (enum: positive, negative, mixed). Testing reveals two issues with brief or ambiguous reviews (~20% of the dataset): (1) for reviews like "Great product!", Claude fabricates specific pros and cons rather than indicating this information isn't explicitly stated, and (2) for sarcastic reviews like "Well that was... interesting", Claude picks sentiment arbitrarily since there's no option for ambiguous cases. What schema modification best addresses both issues?

A. Add an extraction_confidence field (0.0-1.0) for each value, and filter outputs where any confidence falls below a threshold.

B. Allow null values for pros/cons, and add "unclear" to the sentiment enum.

C. Allow empty arrays for pros/cons as valid output, and add "unclear" to the sentiment enum.

D. Make pros and cons optional fields, and add "neutral" and "unclear" to the sentiment enum.

Question: 118 CertyIQ

After your daily batch of 10,000 documents completes, 300 documents (3%) failed with "context_length_exceeded" errors. The results file identifies each failure by custom_id. What's the most cost-effective approach to process these failures?

A. Reprocess the entire batch with prompt caching enabled to reduce the cost of retrying requests with identical system prompts

B. Resubmit only the 300 failed documents after chunking them into smaller pieces, then combine the partial extractions

C. Increase the max_tokens parameter for the 300 failed documents and resubmit them in a new batch

D. Resubmit the entire 10,000 document batch using a model tier with a larger context window

Question: 119 CertyIQ

The system needs to extract candidate information (name, contact details, skills, work experience, education) from uploaded resumes. The extracted data must strictly conform to a predefined JSON schema, as missing required fields or incorrect data types will cause downstream validation failures. What is the most reliable approach to ensure Claude's output consistently matches the schema?

A. Define a tool with an input schema matching your required JSON structure and extract the data from Claude's tool_use response.

B. Include detailed JSON formatting instructions and a template example in the system prompt, asking Claude to output only valid JSON.

C. Parse Claude's text response with regex patterns to extract JSON objects, using retry logic for malformed responses.

D. Make two separate API calls — first extracting information as text, then asking Claude to format that text as JSON.

Question: 120 CertyIQ

After deploying automated code review, developers report that approximately 35% of flagged findings are false positives falling into consistent patterns: style suggestion contradicting team conventions, security warnings for patterns safe in your deployment context, and performance suggestions that would degrade your specific use ca. You want to reduce false positives while maintaining the ability to catch genuine issues. Which approach best enables the model to generalize its judgment to novel co patterns it hasn't seen before?

A. Include few-shot examples in your prompt showing annotated code snippets that distinguish acceptable patterns from genuine issues in each category.

B. Implement post-processing that uses keyword matching to filter out findings containing terms like "convention," "context-dependent," or "trade-off."

C. Add instructions to your system prompt to "be conservative," "only flag definite issues," and "consider that some patterns may be intentional."

D. Create a comprehensive written specification of all patterns that should not be flagged, then include this full documentation in the system prompt.

Question: 121 CertyIQ

After expanding the agent's MCP tools with delivery-specific capabilities (check_delivery_status, contact_driver, issue_credit, apply_promo_code, update_delivery_address, reschedule_delivery), the total tool count has grown

from 4 to 10. Your evaluation suite shows tool selection accuracy has dropped to 71%. Log analysis reveals the majority of errors involve the agent selecting between semantically overlapping tools-calling issue_credit when process_refund is correct, and calling check_delivery_status when lookup_order already returns the needed data. Which approach structurally eliminates the semantic overlaps that are being logged as the error source?

A. Split the tools across two sub-agents - a "financial resolution" agent with process_refund, issue_credit, and apply_promo_code, and a "delivery" agent with the remaining delivery tools - with a coordinator routing between them.

B. Add few-shot examples to the system prompt demonstrating correct selection for each ambiguous tool pair, such as showing when issue_credit or process_refund is appropriate.

C. Consolidate semantically overlapping tools-merge issue_credit and process_refund into a single resolve_compensation tool with an optional include_tracking flag.

D. Enable the tool search tool with defer_loading on the six new tools, keeping the original four always loaded, so the agent dynamically calls it when needed.

Question: 122 CertyIQ

Anthropic's tool use documentation states: "Write instructive error messages. Instead of generic errors like 'failed', include what went wrong and what Claude should try next." A billing dispute agent uses lookup_order, which catches all exceptions and returns a tool_result with is_error: true and the message "execution failed". Monitoring shows two failure modes: the agent retries the identical call until hitting the turn limit, or it immediately calls escalate_to_human without trying alternative tools. Which change follows the documented recommendation and gives Claude the information it needs to select the correct recovery action for each error type?

A. Implement retry logic with exponential backoff inside each tool implementation so transient errors are resolved transparently within the tool before any failure result is surfaced to Claude in the agentic loop.

B. Return error-type-specific messages with is_error: true, e.g., "order not found-try get_customer to search by phone" for data errors and "Database timeout (transient)-retry should succeed" for infrastructure errors.

C. Remove is_error: true and return the error details as normal tool content, so Claude reasons about the response as data rather than treating it as a flagged failure condition that biases retry behavior.

D. Add an error classification step in the agentic loop that intercepts tool errors before Claude sees them, tags each as "retry" "try_alternative," or "escalate," and adds that recommendation to the tool result.

Question: 123 CertyIQ

Your code review prompts include both implementation changes and the corresponding test file, but the LLM's review comments fail to point out untested code paths.
Analysis reveals the model correctly flags functions that have no tests at all, but fails to identify when conditional branches or error-handling paths within tested functions that have no tests at all, but fails to identify when conditional branches or error-handling paths within tested functio
lack coverage.
What's the most effective way to improve detection of branch-level coverage gaps without overcomplicating the pipeline?

A. Implement a multi-pass pipeline where separate LLM calls first extract all conditional branches, then cross-reference each against test assertions in a second pass.

B. Include few-shot examples showing code with an uncovered branch paired with the review comment identifying the specific missing test case.

C. Add explicit instructions directing the model to enumerate each conditional branch and exception path, then verify each has a corresponding test assertion.

D. Restructure the prompt to interleave implementation and tests, presenting each function followed immediately by its test cases

Question: 124 CertyIQ

After deploying the automated review, you notice high precision but low recall - real bugs are slipping through undetected. Investigation reveals your review prompt instructs Claude to "only report high-confidence issues you are certain about" and "err on the side of not commenting." Developers appreciate the low noise, but a race condition that caused a production outage was visible in a reviewed PR and went unreported. You need to substantially improve bug detection while keeping false positive rates manageable for your team. What is the most effective approach?

A. Expand the context window by including related test files, recent git history, and the module's dependency graph alongside the diff, giving Claude richer signals to assess issue severity.

B. Remove the conservative filtering instructions and prompt Claude to report all potential issues, then apply a programmatic filter to deduplicate and suppress categories that historically generate false positives.

C. Split the review into a finding stage where Claude's goal is coverage - flagging every potential issue with confidence and severity metadata - and a separate thresholds those findings.

D. Add detailed few-shot examples demonstrating bug categories Claude should flag - race conditions, null dereferences, error handling gaps - while keeping confidence filtering instruction to maintain current precision levels.

Question: 125 CertyIQ

Your automated reviewer uses a single prompt covering security issues, API design, and business logic correctness. Your evaluation suite shows strong recall findings (82%) but poor recall for business logic edge cases in quiz scoring (34%). When you add few-shot examples of logic bugs to the prompt, logic recall is 41% but API design recall drops to 68%. How should you address this trade-off to improve detection across both categories?

A. Split the review into separate focused prompts - one for security and API design, another for business logic - each with dedicated examples, then combine findings before posting.

B. Upgrade to a more capable model tier, since its stronger reasoning will handle both concern types in a single prompt and eliminate the recall trade-off.

C. Provide the full repository as context instead of just the changed files and surrounding code, giving the model deeper visibility into business logic.

D. Replace the few-shot examples with a detailed checklist of specific logic edge cases to verify, such as division-by-zero in score calculation or grading thresholds.

Question: 126 CertyIQ

Your agent is handling a billing dispute. After calling get_customer and lookup_order, it identifies that the dispute involves a promotional pricing ero
approval — beyond the agent's authorization level. How should the workflow handle this mid-process escalation?

A. Persist the complete conversation and tool response history to a database, then call escalate_to_human with a reference ID.

B. Compile a structured handoff with customer details, order info, and the identified issue before calling escalate_to_human

C. Attempt the refund with process refund anyway, escalating only if the system rejects the transaction.

D. Call escalate_to_human passing only the customer's original message.

Question: 127 CertyIQ

Your agent has called lookup_order multiple times while investigating a customer's return requests. Each response includes 40+ fields (items, shipping details, payment info, status history). Tool outputs now represent the majority of the conversation's context. The customer mentions two more orders they want to discuss. What's the effective approach before making additional lookups?97

A. Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details

B. Proceed with additional lookups without modifying the existing tool output context

C. Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues

D. Have the model generate a natural language summary of each order's key details, replacing structured responses with prose descriptions

Question: 128 CertyIQ

Production logs show that when the agent handles complex billing disputes requiring 6+ tool calls, it sometimes exhausts its max_turns limit after gathering data and before completing resolution or escalating. The team's goal is to guarantee that every customer interaction ends with either a completed resolution or a human escalation, regardless of how the agent loop terminates. Which approach achieves this guarantee?

A. Add orchestration-layer code that checks the agent's outcome after each loop termination - if the loop ended without a completed resolution or escalation, programmatically call escalate_to_human with the accumulated conversation context and tool results.

B. Implement a pre-tool-use hook that counts tool invocations and terminates the loop with an automatic escalation once the agent reaches 80% of its remaining actions.

C. Add system prompt instructions telling the agent to call escalate_to_human with a summary of its findings whenever it determines it cannot resolve the dispute.

D. Split the workflow into two sequential agent invocations — a first agent gathers information via get_customer and lookup_order, then the second agent uses that data and handles process_refund or escalate_to_human, each with separate turn budgets.

Question: 129 CertyIQ

The document analysis agent has a single analyze_document tool that takes a document and a free-text instruction parameter. During evaluation, requests like "key financial metrics" often return narrative summaries, while "summarize the methodology" sometimes returns raw data tables. The synthesis agent reports analysis results require re-requests with clarified instructions. What's the most effective way to improve reliability?

A. Enhance the tool description with detailed examples showing how different instruction phrasings should map to different output formats

B. Keep the single tool but add an analysis type enum parameter requiring explicit selection between extraction, summarization, and verification

C. Have the coordinator pre-classify each analysis request before passing instructions to the document analysis agent

D. Split the generic tool into purpose specific tools — extract data points, summarize content, verify claim against source — each with

Question: 130 CertyIQ

In production, you observe that simple fact-checking queries (e.g., "What year was the Paris Climate Agreement signed?") traverse all four subagents sequentially, consuming 40+ seconds and significant tokens per query. Complex comparative research benefits from the full pipeline. Your query distribution is diverse and evolving as users discover new applications. What's the most effective approach to optimize for varying query complexity?

A. Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements.

B. Train a query complexity classifier on labeled historical data to predict optimal subagent combinations, retraining periodically as query patterns evolve.

C. Create a fast-path for factual questions that bypasses subagents entirely, routing all other queries through the complete pipeline to ensure research thoroughness.

D. Implement pattern-based routing that categorizes queries by structure (single-fact vs. comparative vs.

analytical) and maps each category to a predefined combination.

Question: 131 CertyIQ

Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analyzer had partially completed extraction, and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings. What state management approach best balances information fidelity with context efficiency when restoring agent state?

A. Index all agent outputs in a shared vector store. When resuming, each agent queries the store using semantic search to retrieve relevant prior findings.

B. Have each agent maintain its own persistent state file and reload it independently at the start of each session.

C. Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts.

D. Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming.

Question: 132 CertyIQ

Your agent has analyzed a complex service module — reading 23 source files, tracing request flows, and identifying error handling patterns. A developer needs to develop two testing strategies before committing to one: end-to-end tests with mocked external services vs. snapshot tests capturing expected outputs. The developer needs to independently develop both approaches to evaluate trade-offs. How should you manage the sessions?

A. Resume the analysis session with fork_session enabled, creating a separate branch for each testing strategy.

B. Export the analysis session's key findings to a file, then create two new sessions that reference this file.

C. Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially.

D. Start two fresh sessions, having each re-read the relevant source files before beginning.

Question: 133 CertyIQ

Production monitoring shows the research phase takes longer than expected. Analysis reveals the coordinator invokes the web search subagent, then invokes the document analysis subagent and waits again. These tasks are independent — neither requires the other's output. What is the most effective way to run these subagents concurrently?

A. Switch both subagents to use a Haiku-tier model instead of Sonnet to reduce their individual execution time.

B. Create an async orchestration layer outside the agent that spawns parallel threads, each running a separate coordinator.

C. Structure the coordinator to emit both Task tool calls (for web search and document analysis) in a single response message.

D. Add detailed instructions to the coordinator's system prompt explaining the performance benefits of parallel execution at the same time.

Question: 134 CertyIQ

An engineer used the agent yesterday to analyze a legacy authentication module, identifying two distinct refactoring approaches: extracting a microservice versus refactoring in-place. Today, they want to explore both approaches in depth — having the agent propose specific code changes for each — before deciding which to implement. What's the most effective way to structure this exploration?

A. Use fork_session to create two branches from yesterday's analysis, exploring one approach in each fork.

B. Resume yesterday's session and explore both approaches sequentially within the same conversation thread.

C. Resume yesterday's session to explore the first approach, then start a new session for the second, manually recreating the original context.

D. Start two fresh sessions, manually providing a summary of yesterday's analysis findings to establish context.

Question: 135 CertyIQ

An engineer asks your agent to add comprehensive tests to a legacy codebase with 200 files and minimal existing test coverage. The engineer hasn't spent much time on the codebase, so they ask for help identifying which modules to prioritize. How should the agent decompose this open-ended task?

A. Use Glob and Grep to map codebase structure, identify heavily-coupled modules, create a prioritized plan for high-impact areas, and revise as dependencies are discovered.

B. Create a fixed testing schedule upfront based on directory structure, allocating equal effort to each top-level directory regardless of code complexity or importance.

C. Systematically read all 200 files to create a complete function inventory before writing any tests, ensuring the testing plan accounts for everything from the beginning.

D. Start writing tests for the first module alphabetically, using test failures and imports to discover related files organically.

Question: 136 CertyIQ

Your automated review calls the Claude API for each PR, using tool_use with a report_findings tool that returns a JSON array of finding objects (each with file_path, line_number, severity, category, and description). During testing on a large PR touching 30+ files, the response hits the max_tokens limit and the output is truncated mid-JSON, causing your pipeline's parser to fail. What is the most effective way to handle this?

A. Increase max_tokens to the model's maximum and instruct Claude to keep finding descriptions under 50 words each.

B. Switch from tool_use to prompting Claude to return findings as a markdown list.

C. Split the review into multiple API calls that each analyze a subset of the changed files, then merge the resulting findings arrays.

D. Add retry logic that detects truncated JSON and re-sends the request with instructions to report only critical and high severity findings.

Question: 137 CertyIQ

You are setting up a non-interactive automated code review pipeline using Claude Code. You want Claude to analyze a pulled Git diff (git diff) against the main branch and apply a custom set of code review instructions. However, you notice that when you run the pipeline, Claude only looks at the raw diff text itself and completely stops using its file-reading or code navigation tools. As a result, it fails to inspect the broader codebase repository context, which is critical because the diff modifies a core function called by many other external modules.
Which change to the CLI invocation will cause Claude to read related files in the repository while still successfully applying your custom review instructions?

A. Replace --system-prompt with --append-system-prompt so your review instructions are added to Claude Code's default prompt instead of overwriting the built-in guidance for using file-reading and code navigation tools.

B. Keep --system-prompt and add --allowedTools "Read, Glob, Grep" so that the non-interactive mode permits file system tools that it otherwise disables.

C. Stop piping the diff via stdin and instead embed the diff contents inside the prompt string, so Claude Code treats the invocation as an agentic session rather than a stream-processing one.

D. Remove --system-prompt entirely and place the review instructions in a CLAUDE.md file at the repo root, since --system-prompt is incompatible with tool use under -p.

Question: 138 CertyIQ

Your development team is using Claude Code to automate test generation across a large codebase. However, developers are frequently rejecting the generated test suites because Claude creates a high volume of trivial assertions or tests that merely maximize line coverage without validating meaningful behavioral logic or edge cases.
You want to guide Claude to generate high-quality, production-ready tests directly without introducing high latency or modifying the core pipeline script.
Which strategy best ensures that high-quality, meaningful tests are generated in the first place?

A. Restrict test generation to directories where historical quality metrics show higher acceptance rates, disabling it for areas where generated tests consistently require heavy editing.

B. Add post-generation coverage analysis that automatically filters out any generated test that doesn't increase line coverage beyond what existing tests provide.

C. Document testing standards in CLAUDE.md including valuable test criteria, available fixtures with intended use cases, and examples distinguishing meaningful behavioral tests from trivial assertions.

D. Implement a two-phase generation where a second Claude call scores each test against quality criteria, filtering out low-scoring tests before presenting results to developers.

Question: 139 CertyIQ

A developer uses Claude Code to refactor a function during their development session. Before committing, they ask the same Claude session to review the code for issues. Later, a separate automated CI review catches several bugs that the same-session review missed. What best explains this discrepancy?

A. The CI environment has access to the full codebase context while the local session only sees the current file

B. The CI review uses a more specific prompt tailored for catching bugs, while the developer's request was too general

C. Claude retains context about its prior reasoning in the session, making it less likely to question its own decisions

D. The extended session length caused the context window to fill with conversation history, leaving less room for thorough analysis

Question: 140 CertyIQ

Your automated reviewer uses a single prompt covering security issues, API design, and business logic correctness. Your evaluation suite shows strong recall for API design findings (82%) but poor recall for business logic edge cases in quiz scoring (34%). When you add few-shot examples of logic bugs to the prompt, logic recall improves to 41% but API design recall drops to 68%. How should you address this trade-off to improve detection across both categories?

A. Split the review into separate focused prompts — one for security and API design, another for business logic — each with dedicated examples, then consolidate findings before posting.

B. Replace the few-shot examples with a detailed checklist of specific logic edge cases to verify, such as division-by-zero In score calculations and boundary conditions in grading thresholds.

C. Upgrade to a more capable model tier, since its stronger reasoning will handle both concern types in a single prompt and eliminate the recall trade-off.

Question: 141 CertyIQ

Your codebase exploration tool stores session IDs to allow engineers to continue investigations across work sessions. An engineer spent an hour yesterday analyzing a legacy authentication module, building context about its architecture and dependencies. They want to continue today. The session ID is valid, but version control shows 3 of the 12 files the agent previously read were modified overnight by a teammate's merge. What approach best balances efficiency and accuracy?

-

A. Resume the session and immediately have the agent re-read all 12 previously analyzed files to ensure complete alignment.

B. Resume the session without informing the agent about the changed files, allowing it to naturally detect discrepancies during execution.

C. Resume the session and inform the agent which specific files changed for targeted re-analysis while retaining context on the unchanged files.

D. Start a fresh session to ensure the agent works with current codebase state without stale assumptions from the previous day.

Question: 142 CertyIQ

An engineer sees an unfamiliar error message "SYNC_CONFLICT: entity version mismatch detected" in production logs but doesn't know which of the 12 services in the codebase generates it. They ask the agent to help locate the source code. What exploration approach will most efficiently find the responsible code?

A. Use Grep to search for distinctive text from the error message (like "SYNC_CONFLICT" or "entity version mismatch"), then Read the matching files to understand context.

B. Use Glob to find files in directories commonly associated with error handling (such as errors/, exceptions/, or handlers/) across services, then Read each matching file.

C. Use Grep to find all files that import the project's error handling module, then Read those files to locate custom error definitions.

D. Read the project's README and service configuration files to understand the architecture, then systematically Read source files in service directory.

Question: 143 CertyIQ

You've configured your Claude agent with three MCP servers: one for git operations, one for Jira ticket management, and one for documentation search. When a user asks the agent to "create a branch for JIRA-123 and add documentation links to the ticket," how does the agent access tools across these servers?

A. The agent queries each server sequentially to determine which handles each tool, routing calls based on tool name prefixes.

B. Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent.

C. You must specify which MCP server to use for each turn, and the agent can only access one server's tools at a time.

D. The agent automatically selects the most relevant server based on the request and loads only that server's tools.

Question: 144 CertyIQ

Compliance requires that refunds exceeding $500 must automatically escalate to a human agent-this rule cannot be left to model discretio Despite clear system prompt instructions, production logs show the agent occasionally processes high-value refunds directly (3% failure rate) How should you achieve guaranteed compliance?

A. Modify the refund tool to return an error with message "Amount exceeds policy limit please escalate" when threshold is exceeded.

B. Implement a hook to intercept tool calls when the refund process amount exceeds $500, block it and invoke human escalation.

C. Strengthen the system prompt with emphatic language: "CRITICAL POLICY! Refunds over $500 MUST trigger human escalation. NEVER process these directly!"

D. Add few-shot examples to the prompt showing correct escalation behavior at various refund amounts ($40, $500, $600).

Question: 145 CertyIQ

Production logs reveal inconsistent error handling: when tool_code fails, the agent sometimes retries 5 times (even if the tool_id doesn't exist), sometimes escalates immediately (premature for temporary network issues), and sometimes adds user-friendly explanation (inappropriate when the issue is a backend permission error). Investigation shows four MCP tool returns uniform error responses: "status": "error", "content": " "type": "Error", "message": "Operation failed." " . The agent learns different types.
What's the most effective improvement?

A. Implement retry logic with exponential backoff in your MCP server for all errors, returning to the agent only after retries are exhausted.

B. Create an analyze_error MCP tool the agent calls after any failure to determine the error category and recommended action.

C. Add a few-shot examples to the system prompt demonstrating how to interpret error message patterns and select appropriate responses for each.

D. Enhance error responses with structured metadata. Include error_category (transient/retriable/permission), reason, and a description of what caused the failure.

Question: 146 CertyIQ

Production logs reveal inconsistent error handling. When looking up facts, the agent sometimes retries 5+ times (sadly, when the outer ID doesn't exist), sometimes escalates immediately (prematurely for temporary network issues), and sometimes asks users for clarification (inappropriate when the issue is a backend permission error). Investigation shows your MCP tool returns uniform error responses ("isError": true, "context": "type": "text", "text": "Operation failed."). The agent cannot distinguish between error types.
What's the most effective improvement?

A. Implement retry logic with exponential backoff in your MCP service for all errors returning to the agent only after retries are exhausted.

B. Create an analyze_error MCP tool that the agent calls after any failure to determine the error category and recommended action.

C. Add ten shot examples to the system's conduct_demo training to interpret error message context and select appropriate responses for each.

D. Return structured error metadata in MCP responses.

Question: 147 CertyIQ

After expanding the agent's MCP tools with delivery-specific capabilities [apply_promo_code, update_delivery_address, reconcile_delivery], the total tool count has grown from 1 to 7. We have observed that the agent now shows tool selection accuracy has dropped from 86% to 71%. Log analysis reveals the majority of errors involve the agent selecting between semantically overlapping tools — calling issue_credit when process_refund was correct, and calling check_delivery_status when looking order_allready_returns_the_needed_data. Which approach structurally eliminate the semantic overlap identified in the error source?

A. Consolidate semantically overlapping tools – merge issue_credit and process_refund into a single handle_promotions tool with an action parameter and fold check_delivery_status into lookup_order with an optional include_tracking flag.

B. Enable the tool search tool with defer_loading on the six new tools, keeping the original two always loaded, so the agent dynamically discovers specialized tools only when needed.

C. Add few-shot examples to the system prompt demonstrating correct selection for each ambiguous tool, such as showing when issue_credit applies versus when process_refund is appropriate.

D. Split the tools across two sub-agents — a 'financial resolution' agent with issue_credit, process_refund, return_order, and apply_promo_code and a 'delivery operations' agent with the remaining delivery tools — with a coordinating routing between them.

Question: 148 CertyIQ

You're implementing the escalation logic for when the agent should call a human. Your team processes for different approaches for triggering escalation. Which approach will most reliably identify cases that genuinely require human intervention?

A. Instruct the agent to escalate when the customer requests a human. When the issue requires policy exceptions, or the agent can not make meaningful progress.

B. Configure the agent to escalate after three consecutive tools fails to resolve the customer's stated issue, ensuring a reasonable attempt before involving a human.

C. Build a rules engine that maps specific issue types, customer segments and product categories to escalation decisions, removing the need for model judgment calls.

D. Implement sentiment analysis that monitors for frustration indicators (e.g., all-caps, repeated questions, exclamations) and trigger escalation when the frustration score exceeds a configured threshold.

Question: 149 CertyIQ

Your code-review prompts include both implementation changes and the corresponding test file, but the review comments fail to identify untested code paths. The model correctly flags functions that have no tests at all, but it fails to recognize when conditional branches or error-handling paths within tested functions lack coverage.
What is the most effective way to improve branch-level gap detection without overcomplicating the pipeline?

A. Interleave the implementation and tests in the prompt, presenting each function immediately before its test cases.

B. Add explicit instructions requiring Claude to enumerate every conditional branch and exception path, then verify that each path has a corresponding test assertion.

C. Implement a two-pass pipeline in which one model call extracts all conditional branches and another cross-references them against test assertions.

D. Include few-shot examples showing code with an uncovered branch and the corresponding review comment identifying the missing test case.

Question: 150 CertyIQ

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.
Your automated review jobs take 18 seconds to initialize before Claude begins analyzing code. Profiling reveals that the delay results from automatically discovering hooks, MCP servers, plugins, skills, and multiple nested CLAUDE.md files throughout the monorepo.
You need to reduce startup time while ensuring reviews still enforce the coding standards documented in the root-level CLAUDE.md file.
What is the most effective approach?

A. Replace the default prompt using --system-prompt-file ./CLAUDE.md, which bypasses default prompt assembly and loads only the project rules.

B. Run in --bare mode and pass --append-system-prompt-file ./CLAUDE.md to load the required project standards explicitly while skipping automatic discovery.

C. Run in --bare mode and repeat all review criteria directly in the -p prompt for every invocation.

D. Keep the default initialization and add --exclude-dynamic-system-prompt-sections to improve prompt-cache reuse across CI runners.

Question: 151 CertyIQ

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.
An engineer submits two requests:
Request A: “Rename the getUserData function to fetchUserProfile everywhere it’s used.”
Request B: “Improve error handling throughout the data processing module — add try/catch blocks, meaningful error messages, and ensure failures don’t silently corrupt data.”
For which request does specifying an explicit multi-phase workflow (such as analyze → propose → implement with review) most improve outcome quality?

A. Neither request benefits significantly

B. Request A, the function rename task

C. Both requests benefit equally

D. Request B, the error handling task

Question: 152 CertyIQ

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.
A developer uses Claude Code to refactor a function during a development session. Before committing, the developer asks the same Claude session to review the code for issues. Later, a separate automated CI review catches several bugs that the same-session review missed.
What best explains this discrepancy?

A. Claude retains the implementation context and prior decisions in the session, making it less likely to challenge assumptions underlying its own changes.

B. The session’s context window necessarily became full, leaving insufficient capacity for meaningful review.

C. The CI review must have used a more specific prompt, while the developer’s review request was too general.

D. The CI environment can access the full repository, while a local Claude Code session can access only the current file.

Question: 153 CertyIQ

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.
Your pipeline includes a release-notes generation step that classifies and summarizes approximately 200 commits at the end of each weekly release cycle. Each commit is currently sent as a separate Messages API request using a Sonnet-tier Claude model. The release notes are not needed until the following morning, providing approximately 12 hours of acceptable latency.
Your team must reduce the per-token API cost while retaining the same model, prompts, and output quality.
Which approach satisfies all these constraints?

A. Issue the 200 Messages API requests concurrently because parallel execution reduces the per-token price.

B. Submit the 200 requests through the Message Batches API with unique custom_id values and retrieve the results after the batch finishes.

C. Concatenate all 200 commit messages into one Messages API request because reducing the number of requests always reduces token costs.

D. Replace the Sonnet-tier model with a Haiku-tier model to obtain a lower per-token price.

Question: 154 CertyIQ

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.
Your extraction system processes two document types: standard monthly reports, which are archived after processing, and urgent exception reports, which must trigger business alerts within 30 minutes of receipt. Both use the same JSON schema. You want to minimize API costs while meeting the latency requirements.
How should you architect the processing pipeline?

A. Submit all documents to the Message Batches API with custom_id values for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.

B. Route standard reports to the Message Batches API for 50% cost savings, and route urgent exception reports to the real-time Messages API.

C. Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.

D. Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.

Question: 155 CertyIQ

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.
Testing reveals that when source documents are missing certain specifications, the model fabricates plausible-sounding values to satisfy your schema's required fields. For example, a document mentioning only dimensions receives a fabricated "weight: 2.3 kg" in the extraction output.
What schema design change most effectively addresses this hallucination behavior?

A. Add explicit instructions to the prompt stating "only extract information explicitly stated in the document; use placeholder text for missing values."

B. Change fields that may not exist in source documents from required to optional, allowing the model to omit them.

C. Add a "confidence" field alongside each specification where the model self-reports its certainty, then filter out low-confidence extractions.

D. Implement semantic validation that verifies each extracted value appears in or can be inferred from the source document text.

Question: 156 CertyIQ

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.
Your extraction pipeline occasionally receives responses that cannot be parsed as valid JSON, causing downstream processing failures. The current implementation prompts Claude to return JSON in the response text and then parses it.
What is the most reliable approach to ensure Claude returns valid, schema-compliant structured data?

A. Add explicit formatting instructions to the prompt with JSON examples, emphasizing that Claude must return only valid JSON with no surrounding text.

B. Use regular expressions to locate and extract JSON from the response text, handling cases where Claude includes explanatory text around the JSON block.

C. Define a tool with a JSON schema specifying the expected structure, using tool use to constrain Claude's

output to schema-compliant JSON.

D. Implement a retry loop that catches JSON parsing errors and re-prompts Claude with the error details, asking it to correct the malformed output.

Question: 157 CertyIQ

You are building developer-productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools — Read, Write, Bash, Grep, and Glob — and integrates with Model Context Protocol (MCP) servers.
Your productivity agent connects to three MCP servers: an issue tracker with search_issues , get_issue , and create_comment ; a documentation wiki with search_docs , get_page , and list_spaces ; and a database explorer with run_query , get_schema , and list_databases . For cross-system questions such as, “Which database tables are affected by the authentication refactor in PROJ-1234?”, the agent makes eight to ten sequential exploratory calls, lacks visibility into each server’s available content, and exhausts context before completing complex investigations.
What architectural change best leverages MCP capabilities to address these problems?

A. Add a prepare_investigation tool to each server that accepts a natural-language question and returns relevant summaries.

B. Consolidate all three systems into one unified MCP server with cross-referencing capabilities.

C. Expose each server’s content catalog as MCP resources, including issue summaries, documentation hierarchies, and database schemas.

D. Add an orchestrator that routes each question to one server based on keywords.

Question: 158 CertyIQ

The coordinator provides detailed step-by-step instructions to the web-search subagent, specifying exact search queries, source priorities, and date filters. Production monitoring reveals three issues: (1) the subagent reports “insufficient results” instead of trying alternative approaches when the specified searches fail, (2) research quality drops for emerging topics that do not match expected patterns, and (3) the subagent rarely surfaces valuable tangential sources. What is the most effective way to improve subagent adaptability?

A. Specify research objectives and quality criteria — such as coverage breadth, source diversity, and recency — rather than prescribing procedural steps, allowing the subagent to determine its search strategy.

B. Remove procedural details entirely and delegate using simple goals such as “research this topic thoroughly,” relying on the subagent’s general capabilities.

C. Add fallback directives requiring alternative query formulations whenever the specified searches produce fewer than a predetermined number of results.

D. Classify each topic as either “well-defined” or “exploratory” and use a different instruction style for each category.

Question: 159 CertyIQ

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web-search and document-analysis agents did not find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

A. Have the coordinator evaluate the synthesis output for gaps, then redelegate targeted queries to the web-search and document-analysis agents before invoking synthesis again.

B. Have the report-generation agent identify unanswered research questions so users understand the limitations of the final output.

C. Increase the initial breadth of queries sent to the web-search and document-analysis agents to reduce the probability of missing relevant information.

D. Give the synthesis agent direct access to web-search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.

Question: 160 CertyIQ

In addition to your CI pipeline, your organization has enabled Claude's managed Code Review through the Claude GitHub App on this repository, and reviews run automatically on every pull request. Reviews average 18 findings per pull request. Developer feedback reveals three categories of unwanted noise: (1) style and formatting issues already enforced by your CI linter, (2) findings on automatically generated template code under src/gen/*, and (3) rendering-helper patterns that are intentional project conventions but are flagged because they resemble common anti-patterns. Only approximately four findings per pull request are genuine logic bugs.
What is the most effective way to reduce this noise while preserving the detection of real issues?

A. Create a REVIEW.md file at the repository root containing skip rules for CI-enforced checks and generated files, together with a verification requirement that rendering-related findings cite a specific line demonstrating incorrect behavior.

B. Configure separate GitHub Actions workflow files for each code area: one for generated code with findings suppressed, one for rendering code with custom instructions, and one general workflow for everything else.

C. Add custom review instructions to a GitHub Actions workflow file, using the action's prompt parameter to suppress duplicate lint findings, ignore generated template code, and impose stricter evidence requirements on rendering-related issues.

D. Add detailed explanations to the project's CLAUDE.md describing intentional patterns, stating that CI handles linting, and identifying src/gen/ as automatically generated code.

Question: 161 CertyIQ

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.
Your team has connected a custom MCP server that provides DevOps workflow templates. The server exposes several MCP prompts (such as deploy_checklist and incident_response) in addition to tools.
How do these MCP prompts become accessible within Claude Code?

A. They are automatically prepended to every conversation as additional system-level context, influencing Claude's behavior throughout the session.

B. They are added to Claude Code's tool registry alongside the server's tools, invoked automatically by the model when relevant to the task.

C. They are surfaced as @ -mentionable resources alongside files, fetched and attached to your message when referenced.

D. They appear as slash commands (e.g., /mcp__servername__deploy_checklist) that you can invoke, with arguments passed after the command name.

Question: 162 CertyIQ

Your pipeline includes a release-notes generation step that classifies and summarizes approximately 200 commits at the end of each weekly release cycle. Each commit is currently sent as a separate Messages API call using a Sonnet-tier Claude model. The release notes are not needed until the following morning, so results have approximately 12 hours of acceptable latency. Your team needs to reduce per-token API cost for this step while keeping the same model and prompts, with no change to the model tier or output quality.
Which approach satisfies all these constraints?

A. Concatenate all 200 commit messages into a single Messages API request and have the model return all summaries in one response, because fewer requests always reduce total token cost.

B. Issue the 200 Messages API requests in parallel using concurrent connections, because concurrency lowers the per-token price charged by the API.

C. Submit the 200 requests to the Message Batches API with unique custom_id values and retrieve the results after the batch finishes, which applies a 50% discount to all input and output tokens.

D. Switch the summarization calls from the Sonnet-tier model to a Haiku-tier model to take advantage of Haiku's lower per-token rates.

Question: 163 CertyIQ

You are building developer-productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools — Read, Write, Bash, Grep, and Glob — and integrates with Model Context Protocol (MCP) servers.
Engineers frequently ask the agent to cross-reference code changes with Jira tickets during reviews — checking ticket descriptions, acceptance criteria, and recent comments. This currently requires manually copying and pasting content into conversations. The team wants the agent to access this standard Jira ticket data directly.
What is the most effective approach?

A. Use the Bash tool with curl to call Jira's REST API, including authentication headers and parsing JSON responses inline.

B. Build a custom MCP server wrapping Jira's API with tools designed specifically for this team's code-review workflow.

C. Export Jira tickets to Markdown files in the repository that the agent accesses using the Read tool.

D. Integrate an existing Jira MCP server that exposes tickets, comments, and metadata through discoverable tool interfaces.

Question: 164 CertyIQ

A user expands the research system beyond its original web-search agent by adding specialized data sources. A financial API agent returns structured JSON containing revenue, margins, and growth rates. A news-monitoring agent returns prose summaries of recent developments. A patent-analysis agent returns structured lists of technology areas.
The synthesis agent combines these results into executive briefings. Currently, it converts everything into bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose their narrative flow.
What change would most improve briefing quality?

A. Standardize all subagent outputs as prose summaries with inline citations.

B. Standardize all subagent outputs as JSON containing fields for claim, evidence, source, and confidence.

C. Update the synthesis agent to render each content type appropriately — financial data as tables, news as prose, and technology areas as structured lists.

D. Add a format-conversion layer that transforms every subagent result into a common intermediate representation before synthesis.

Question: 165 CertyIQ

Users report that final reports sometimes lack depth on specific subtopics. Investigation shows that the document-analysis agent frequently identifies evidence gaps — for example, noting that "the retrieved sources discuss API authentication but lack details about token-refresh patterns."
Under the current strict pipeline, this insight is not actionable because searching has already finished.
What is the most effective architectural change?

A. Add a research-planning agent before the initial search phase to decompose every topic into detailed subquestions.

B. Have the synthesis agent assign confidence scores to each report section and flag insufficiently supported sections for manual review.

C. Require the analysis agent to return specific evidence gaps to the coordinator, which launches targeted searches and invokes analysis again until the defined coverage criteria are satisfied

D. Have the coordinator look for general gap indicators in the analysis output and run additional searches without repeating the analysis stage.

Question: 166 CertyIQ

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer , lookup_order , process_refund , escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.
Your process_refund tool returns two types of errors: technical errors ("503 Service Unavailable", "Connection timeout") that are transient (~5% of calls), and business errors ("Order exceeds 30-day return window", "Item already refunded") that are permanent (~12% of calls). Monitoring shows the agent wastes 3–4 turns retrying business errors that can never succeed. Currently, both error types return only a plain text message to Claude.
What's the most effective way to reduce wasted retries while improving customer-facing response quality?

A. Implement automatic retry logic at the tool layer for technical errors only, passing business errors to Claude without retries.

B. Add few-shot examples showing how to distinguish retriable from non-retriable errors by parsing error message text.

C. Add a check_refund_eligibility tool that must be called before process_refund to prevent business rule violations.

D. Return structured error responses with " retriable " : false for business errors and a customer-friendly explanation for Claude to use.

Question: 167 CertyIQ

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.
During initial testing of the automated review pipeline, you notice that reviews on large pull requests containing more than 50 changed files sometimes take over 20 minutes and cost $8–$12 per run because of extensive agentic loops. Claude reads files, runs analysis tools, and iterates many times. Your team needs each invocation to abort once it reaches both a fixed iteration count and a fixed dollar amount, enforced by Claude Code itself rather than by the surrounding job runner.
Which configuration change directly enforces both per-invocation caps?

A. Switch the --model flag to a smaller, less expensive model so each iteration uses fewer tokens and has a lower per-call cost.

B. Set timeout-minutes: 5 on the GitHub Actions job step and monitor per-run costs through the Anthropic Console usage dashboard.

C. Add --max-turns 10 --max-budget-usd 2.00 to the claude -p invocation to cap iterations and spending.

D. Set --permission-mode dontAsk to automatically deny tool-permission requests not included in the explicitly allowed set.

Question: 168 CertyIQ

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.
Your code review assistant needs to analyze pull requests and provide feedback on three aspects: code style compliance, potential security issues, and documentation completeness. Each aspect requires reading files, running analysis tools, and generating a report section. The review process follows the same three-step workflow for every PR.
Which task decomposition pattern is most appropriate for this workflow?

A. Single comprehensive prompt — include all three instructions in one prompt and let the model handle all

three aspects simultaneously.

B. Orchestrator-workers — have a central LLM analyze each PR to dynamically determine which checks are needed, then delegate to specialized worker LLMs for each identified subtask.

C. Prompt chaining — break the review into sequential steps where each aspect (style, security, documentation) is analyzed separately, with outputs combined in a final synthesis step.

D. Routing — classify each PR by type (feature, bugfix, refactor) first, then route to different review prompts optimized for that category.

Question: 169 CertyIQ

An AI agent running in Claude Code (or similar agentic harness with tools like Read, Write, Bash, Grep, Glob, and Model Context Protocol servers) is investigating a large payment processing module spanning 45 files. After reading the first 8 source files, the agent's responses are becoming noticeably less accurate — it is forgetting previously discussed code patterns and hasn't yet located all test files or traced critical payment flows.
What is the most effective approach to complete this investigation without exceeding context limits or causing context degradation?

A. Clear context with /clear, then selectively re-read only the most critical files discovered so far, writing key findings to a scratchpad file that persists between context resets.

B. Document all current findings in a summary report, clear context completely, then use that report as the sole reference for continuing the investigation.

C. Spawn subagents (or sub-tasks) to investigate specific questions (e.g., "find all test files for payment processing," "trace refund flow dependencies") while the main agent coordinates findings and preserves high-level context.

D. Continue reading the remaining files in the main session, relying on Claude's extended context window to auto-truncate earlier messages as new files are loaded.

Question: 170 CertyIQ

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks using built-in tools (Read, Write, Bash, Grep, Glob) and Model Context Protocol (MCP) servers.
Your agent has spent significant context exploring the rendering subsystem — reading shader code, buffer management, and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference "typical rendering patterns" rather than the specific VulkanPipeline and FrameGraph classes it discovered earlier.
What's the most effective approach to handle this transition while preserving accuracy?

A. Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.

B. Use /clear to reset context completely, then start fresh with physics exploration using file paths from the project's CLAUDE.md.

C. Continue in the current context with more targeted prompts referencing the specific classes by name.

D. Summarize key rendering findings (including specific class names and architectural patterns), clear or reset the main context window (or delegate to a sub-agent seeded with that summary), then proceed with the physics integration exploration.

Question: 171 CertyIQ

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks using built-in tools (Read, Write, Bash, Grep, Glob) and Model Context Protocol (MCP) servers.
Your agent has spent significant context exploring the rendering subsystem — reading shader code, buffer management, and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference "typical rendering

patterns" rather than the specific VulkanPipeline and FrameGraph classes it discovered earlier.
What's the most effective approach to handle this transition while preserving accuracy?

A. Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.

B. Use /clear to reset context completely, then start fresh with physics exploration using file paths from the project's CLAUDE.md.

C. Continue in the current context with more targeted prompts referencing the specific classes by name.

D. Summarize key rendering findings (including specific class names and architectural patterns), clear or reset the main context window (or delegate to a sub-agent seeded with that summary), then proceed with the physics integration exploration.

Question: 172 CertyIQ

An engineer's exploration subagent spent 30 minutes analyzing a legacy payment system, reading 47 files and documenting data flows. The session was interrupted when the engineer's connection dropped. While away, a teammate merged a PR that renamed two utility functions. The engineer wants to continue the same exploration.
What is the most effective approach to handle this situation?

A. Launch a fresh subagent and include the prior transcript in the initial prompt for context.

B. Resume the subagent from its previous transcript without mentioning the changes — the architecture understanding remains valid.

C. Launch a fresh subagent with a summary of prior findings.

D. Resume the subagent from its previous transcript and inform it about the renamed functions.

Question: 173 CertyIQ

Your schema includes a 'skills: string[] field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract all skills mentioned."
What's the most effective improvement?

A. Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.

B. Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."

C. Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.

D. Enrich the schema to skill: string, confidence: float, source_quote: string to capture extraction confidence.

Question: 174 CertyIQ

During testing, you find that when a customer says "I need a refund for my recent purchase," the agent calls process_refund immediately – but populates the required 'order_id' parameter with a plausible-looking but fabricated value instead of first calling lookup_order to retrieve the actual order ID. The refund call fails because the fabricated ID doesn't exist.
Which change directly addresses the root cause of the agent fabricating the 'order_id' value?

A. Switch 'tool_choice' from "auto" to "any" to force the agent to make a tool call on every turn.

B. Pre-parse incoming customer messages to extract any order IDs mentioned, and inject them into the conversation context before passing to Claude.

C. Update the 'process_refund' tool description to explicitly state that 'order_id' must be obtained from a prior 'lookup_order' call and must never be assumed or invented.

D. Add server-side validation that checks whether the 'order_id' exists in your database before executing or to

the agent if not.

Question: 175 CertyIQ

A customer returns 4 hours after their initiar session about the same billing dispute. The previous 32-turn session contains lookup_order results showing "Status: PENDING, Expected resolution: 24-48 hours." In testing, you observe that when resuming sessions with stale tool results, the agent often references the outdated data in responses (e.g., "I see your refund is still being processed") even after subsequent fresh tool calls return different information.
What approach most reliably handles returning customers?

A. Resume with full history but filter out previous tool_result messages before resuming, keeping only the human/assistant turns so the agent must re-fetch needed data.

B. Resume with full history and add a system prompt instruction telling the agent to always prefer the most recent tool results when multiple calls to thesame tool exist in context.

C. Start a new session, inject a structured summary of the previous interaction (issue type, actions taken, resolution status), then make fresh tool calls before engaging.

D. Resume with full history and configure the agent to automatically re-call all previously used tools at session start to ensure data freshness.

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!

Feedback

More Papers

Total: **175 Questions**

Link: https://certyiq.com/papers/anthropic/claude-certified-architect